Online regression markets with varying feature space

Author:
Carl Frederik Grønvald
DTU Wind-M-0784
July 2024

Author:
Carl Frederik Grønvald

Title:
Online regression markets with varying feature space

DTU Wind-M-0784
July 2024

ECTS: 32,5

Education: Master of Science

Supervisors:
Jalal Kazempour
Thomas David Falconer
DTU Wind & Energy Systems

Remarks:
This report is submitted as partial fulfillment of the requirements for graduation in the above education at the Technical University of Denmark.

DTU Wind & Energy Systems is a department of the Technical University of Denmark with a unique integration of research, education, innovation and public/private sector consulting in the field of wind energy. Our activities develop new opportunities and technology for the global and Danish exploitation of wind energy. Research focuses on key technical-scientific fields, which are central for the development, innovation and use of wind energy and provides the basis for advanced education at the education.

Technical University of Denmark
Department of Wind and Energy Systems
Frederiksborgvej 399
4000 Roskilde
Denmark
www.vindenergi.dtu.dk

Online Regression Markets with Varying Feature Space

Master Thesis
July, 2024

By
Carl Frederik Grønvald

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Published by: DTU, Department of Wind & Energy Systems, Frederiksborgvej 399, 4000 Roskilde Denmark
www.vindenergi.dtu.dk

Abstract

Data that is useful for a forecasting task is often held privately by different owners in direct or indirect downstream competition. Therefore, these data owners are not willing to share their data freely even if their privacy is preserved. Current literature then proposes analytics markets to provide monetary incentives for data sharing, pricing data according to its value for the buyer's prediction task. In this thesis, we examine analytics markets and motivate them specifically for wind electricity production forecasting. Many current proposals have limitations, specifically they assume a fixed set of features over time. We relax this assumption, reviewing the current literature within online learning with varying feature spaces for both classification and regression and developing a new model for this case known as the Online Linear Model with Subregressions (OLM-S) that reconstructs missing features linearly from present features.

To allocate revenue between market participants, we model features as players and their interactions as a cooperative game. We investigate the outcome of using an allocation policy founded in semivalues from cooperative game theory known as Shapley values in varying feature space. Feature sellers with missing features have a new strategic choice of whether to truthfully report their feature as missing or report a mean imputed value. This gives rise to the property *truthfulness under imputation*. A market that is truthful under imputation incentivizes participants to truthfully report missing features.

We show weak truthfulness under imputation for one market model, but do not show it for the OLM-S model. We test the OLM-S model on a real-world wind energy production dataset, and examine the equilibrium strategy for feature sellers in this market out-of sample. We find them to be truthful under imputation in tested examples.

We conclude that we can use this new OLM-S model for a regression market in a realistic forecasting situation with missing features to incentivize mutually beneficial data exchange and increase society-level social welfare.

The code for the results presented in this thesis can be found at:
https://github.com/carlgronvald/varying_feature_spaces

Acknowledgements

Writing this thesis has been a huge experience for me. At times frustrating, at times boring, at times exciting, and at times intensely rewarding. Thank you to everyone who was there along the way.

Thomas Falconer, PhD Student, Technical University of Denmark
Thank you for supervising me on this thesis and taking so much time out of your days to understand and discuss the problems I faced along the way.

Jalal Kazempour, Associate Professor, Technical University of Denmark
Thank you for supervising my thesis and introducing me to this field of research, as well as prompt and useful feedback and guidance.

Liselotte & Søren Grønvald, My Parents
Thank you for being there for me and listening to me complain whenever things were not going how I wanted, as well as housing me when the city got too loud and I needed some peace and quiet.

Diogo Dantas, My Boyfriend
Thank you for always being supportive and present and dealing with the times where the thesis was all I could think about, as well as the diversion, distraction, and general cozy times that were exactly what I needed when the stress was getting to me.

I also want to thank my brother for sharing his thesis journey with me, my friends for listening and providing input, as well as my boyfriend's family for welcoming me during my stay in Portugal and making it feel like home. Lastly, I want to thank my job for giving me the flexibility and time I needed to write a master's thesis while remaining employed.

Contents

Abstract ii
Acknowledgements iii

1 Introduction **1**
1.1 Data Sharing 1
1.2 Emergence of Data Markets 1
1.3 Motivation for Analytics Markets 2
1.4 Literature Review 6
1.5 Contributions 7

2 Preliminaries **9**
2.1 Notation 9
2.2 Machine Learning 11
2.3 Online Learning 19
2.4 Game Theory 20

3 Varying Feature Spaces and Analytics Markets **27**
3.1 Evolving Feature Space 27
3.2 Market Model Candidates 35
3.3 Revenue Allocation for Missing Features 37

4 Results and Discussion **41**
4.1 Utilitarian Online Learning 41
4.2 Regression 44
4.3 Regression Market 50

5 Conclusions **59**
5.1 Future Work 60

1 Introduction

In the modern world, data is increasingly important for decision making. It is commonly said in machine learning that ”the model is only as good as the data” - a clear message to anyone seeking to improve their data-driven decisions. Interestingly, though, data is often kept private and not exchanged or sold between private actors. This begs the question: If better access to data is valuable and could improve market outcomes, why is this value not extracted? And can anything be done about it?

In this thesis, we dive specifically into renewable electricity forecasting as our motivating reason for the market we construct. Renewable electricity producers use forecasts to accurately bid in the electricity market. In recent years, renewable electricity forecasting has moved towards sophisticated machine learning methods and probabilistic forecasts, as discussed in Hong et al. [2020]. It has been shown across literature that geographically distributed and heterogeneous data sources are useful for improving the quality of renewable production forecasting. [Andrade and Bessa, 2017, Cavalcante et al., 2017] This data is often owned by several different entities, so their results could be improved by increasing the exchange of data. These entities are often in direct or indirect competition with each other, which means that they are not willing to share data even though it would be beneficial to the system as a whole.

1.1 Data Sharing

Since business decisions first started to be made based on machine learning, people started to realize that they could improve their decision-making if they could access the data of others. This inspired early research into what is known as data sharing where multiple parties freely share their data with each other so that all parties can gain from it.

Data sharing was first studied in the context of sharing with no external incentive, e.g. within the field known as federated learning. Federated learning is an approach for training machine learning models in a decentralized manner where private data can be used for training a model without the data itself being shared, thus retaining data privacy, as laid out in Li et al. [2020]. Despite preserving privacy, federated learning is not enough to incentivize data sharing for two central reasons:

1. There is some cost associated with preparing and providing data for sharing which a participant would have to bear altruistically.
2. Data owners might be competitors in a downstream market which might leave any single owner potentially worse off by sharing data, even if everyone was better on average, as shown in Gal-Or [1985]. In the electricity market, for example, improving the prediction of a competitor could potentially leave you worse off as they outperformed you in the market.

1.2 Emergence of Data Markets

To incentivize data sharing, data markets were developed where participants receive compensation for providing their data. Initial data markets were developed as a way to monetize data samples, or what would in tabular data be known as a row of data, as discussed in Balazinska et al. [2011]. This kind of data covers e.g. the information that a social network site has about its customers — the site knows about a number of people and sells

off the information that it has about a single person as a bundle. Examples of this kind of marketplace are the Snowflake or AWS data exchanges. Markets also developed to sell features, what in tabular data would be known as a column, directly as a type of raw data. However, the pricing of both features and samples can be complicated, since it has been shown that the value of data is dependent on the task of the buyer, so the same data can have different value to different buyers. [Cong et al., 2022]

Another kind of data marketplace is the prediction marketplace. This kind of market does not sell data for use in forecasting or decision-making, but instead directly sells predictions. The design of prediction markets was investigated in Chen and Pennock [2010]. These work by giving payout contingent on real-world events, where parties bet on an outcome of an event and get paid based on whether they bet correctly. In this way, prediction markets effectively form financial securities based on real-world events.

1.3 Motivation for Analytics Markets

Let us now consider the electricity market. The electricity market generally works by participants selling and buying electricity contracts a day before the electricity is consumed, known as the day-ahead electricity market. The electricity is then consumed at what is known as imbalance time or real time. Renewable generation depends on weather circumstances, meaning that the actual generation at imbalance time cannot be known but only forecast when selling in the day-ahead market. Let us imagine first a single renewable generating participant in the electricity market, say that it is a wind farm.

We base our analysis on a stylized view of the electricity market. The system operator of the electricity grid maintains a balance between real-time power supply and demand, since electricity cannot be efficiently stored. Any deviation from this balance is known as imbalance. Imbalance must be eliminated by ramping up or down additional resources, which has a cost associated with it. This cost is paid by market participants that deviate from the demand or supply they bid or offered in the day-ahead market in the direction of imbalance. Deviations in the direction of balance will trade out the deviation at the day-ahead spot price, i. e. under-producers will buy energy back at the day-ahead price and over-producers will sell at the day-ahead price. This understanding of the market is based on Aïd [2015] and approximates two-sided balancing markets as seen in France and the Netherlands. We do not consider the intraday market nor futures from before the day-ahead market.

The wind farm must decide how much power to sell in the day-ahead electricity market. The wind farm wants to offer as close as possible to its true generation at imbalance time. If the farm overestimates its production at day-ahead time and sells too much power, it might have to pay for down-regulation in the grid. If it instead underestimates its production at day-ahead time and sells too little power, it will never earn more than the day-ahead price for its surplus power, and might have to pay for up-regulation in the grid. Its revenue is then a function of its prediction accuracy. The single wind farm is illustrated in Figure 1.1, where we see that it bids based on its production forecast, and is paid based on the accuracy of this forecast.

This wind farm makes a multi time horizon prediction based on some data that it has available. Maybe it has some sensors, some current production data, etc. Now, let us say that the wind farm has another competitor in the market — another wind farm. We then have two participants in the market who both will earn more the better their prediction is in our stylized version of the electricity market.

Intuitively, data from competing wind farms should improve forecasting accuracy, which

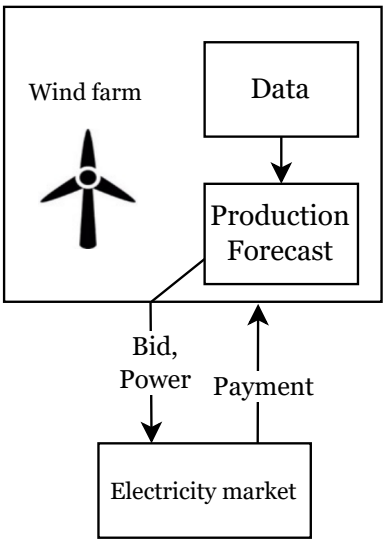


Figure 1.1: A single wind farm in the electricity market, using its own data to forecast its production at imbalance time and provide day-ahead offers based on this. The wind farm is then paid based on the offered power and the electricity price. If the farm over- or undershoots its production, it might have to pay balancing costs for its misprediction, meaning that its profit is based on forecasting accuracy as well.

was also shown in Tastu [2013]. Data exchange between the farms would not only benefit the two wind farms, but also the system operator, as it would have a better incoming prediction of total available generation and thus the total cost of balancing would be lower. This means that the society-level social welfare would be increased if the participants shared data. They might not be willing to share data freely, however, since they are in direct competition in the electricity market. If both wind farms shared data, one wind farm might be more penalized by the improvement in the prediction capability of the other wind farm than they are rewarded by their increase in predictive power. Sharing data with a downstream competitor is not always detrimental as shown in Raith [1996], but data sharing is more likely detrimental when competitors produce highly similar products as shown in Tsoy and Konstantinov [2023], which is the case in the electricity markets. Then, we expect that many renewable electricity producers would not be willing to share data without external incentives.

To incentivize this data exchange, we introduce a new type of data market known as an analytics market. The word analytics market refers to many things in literature but in this thesis it refers to a specific kind of data marketplace, also known as a regression market or classification market. Where earlier data markets traded either 1) data samples, or 2) features, or 3) predictions, sellers in analytics markets sell features indirectly through a central market. A machine learning method then transforms the features into predictions, and these are bought by the buyers. The market structure can be seen in Figure 1.2. Since this requires features to pertain to the same observation or row, the market often deals with time series problems, where sellers provide features for a single time point and buyers buy predictions. This is how the forecasting problem in our motivating example works. A single time step of the analytics market is shown in Figure 1.3. Analytics markets are a fairly new concept and still have a lot of unanswered research questions surrounding them, and are therefore not yet in use in the real world.

Sellers receive payment based on a method from cooperative game theory known as the

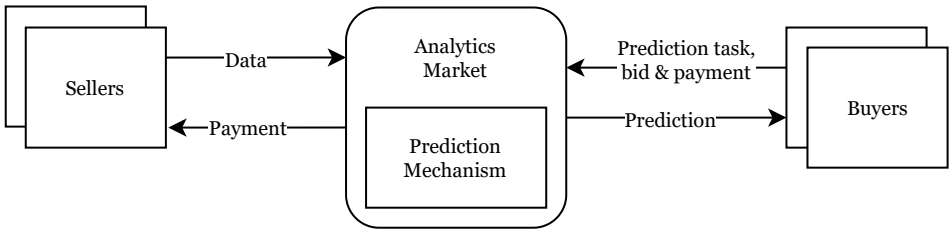


Figure 1.2: The setup of an analytics market. Sellers provide features and receive payment, and buyers provide a supervised learning task and payment and receive a prediction. Their payment is based on predictive performance.

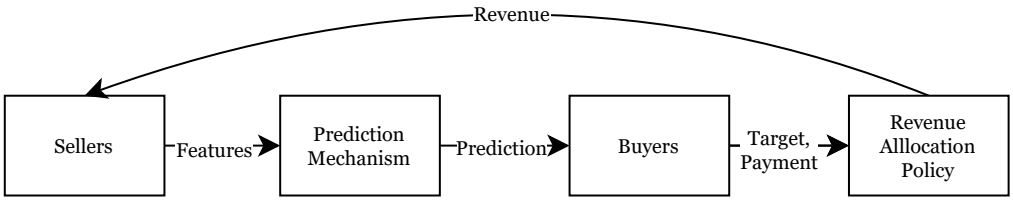


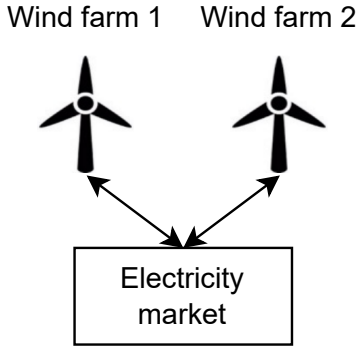
Figure 1.3: A single time step of the analytics market. 1) Sellers provide features 2) These are passed through the machine learning model used by the market, which then forms a prediction 3) This prediction is supplied to buyers, who then give the true regression or classification target along with payment for the predictive improvement 4) Revenue is allocated between features by the allocation policy.

Shapley value. The Shapley value payment allocation is the unique allocation that fulfills a specific set of market properties that are known as the Shapley fairness properties.

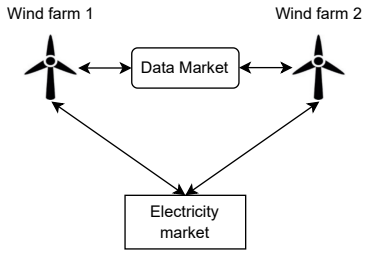
The analytics market provides incentives for data sharing for the participants. They are now paid based on how much their competitor benefits from receiving their data, and everyone will be better off individually. The market facilitates a cooperative environment by aligning incentives between the participating parties. The situation is illustrated in Figure 1.4, showing how adding a market can improve the outcome for all participants.

Analytics market research up until this point has assumed that there is a fixed number of feature sellers in the market that will be present at all market clearings. This is a strong assumption since sellers are individuals or organizations that naturally change their goals and status over time. For example, if a wind farm is temporarily shut off as in Figure 1.5, it would no longer be able to provide production information to a regression market. This thesis seeks to investigate how analytics markets can support a changing number of feature sellers over time, also known as a *varying feature space*.

The main contribution in this thesis is relaxing the assumption of a fixed set of market participants and allowing feature sellers to leave the market or temporarily abstain from providing features, as well as allowing new feature sellers to join the market. This meets a need of the users of analytics markets, and brings these markets closer to being viable in the real world. Relaxing this assumption brings in several new issues for the market: How can revenue be allocated when features can be missing? Which prediction mechanisms can we use and what new requirements will they have? What new risks or opportunities do these markets present both sellers and buyers?

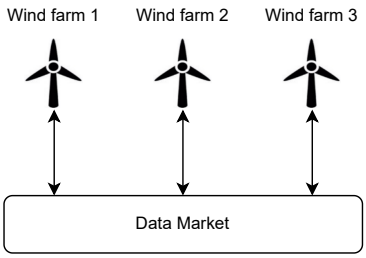


(a) Two wind farms in competition in the electricity market, each bidding individually but wanting each other's data.

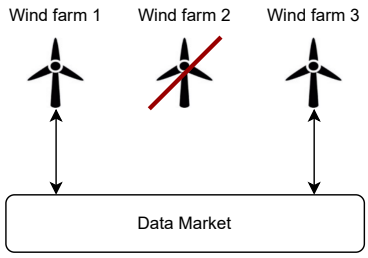


(b) Two wind farms exchanging data through the analytics market, incentivizing sharing and improving societal social welfare.

Figure 1.4: Example showing how two wind farms will not voluntarily share data, but can be incentivized using an analytics market, improving social welfare for both of them as well as the system operator.



(a) At some time step in the market, all three of the shown wind farms are present in the market.



(b) At another time, one wind farm is missing from the market due to unforeseen events or maintenance or similar. The other two wind farms still want to participate.

Figure 1.5: An example that shows how an analytics market can at one time have three present participants but then one can go missing. This is a situation that current literature has not examined.

1.4 Literature Review

A thorough review of the state of the literature surrounding information economics was presented in Bergemann and Bonatti [2019]. It opened up conversation about data markets where participants are in downstream competition, and pointed out that privacy concerns will also be central in these markets. It also emphasized the role of an intermediary for the function of data markets, which aligns with research within analytics markets where an intermediary party, the data market itself, mediates the exchange of information.

The paper Rasouli and Jordan [2021] shows market mechanisms for directly sharing raw data, both in the situation where data is exchanged for data, and where data is exchanged for monetary compensation. They consider privacy and sharing costs in their models, and find data exchange matching algorithms and competitive prices for clearing these market setups. They use distortion of the traded data in the form of noise to control its value.

A regression market is presented in Goncalves et al. [2021] with a case study of a set of wind farms. They value features based on in-sample loss reduction, and show that a regression market can improve social welfare for all participants.

The paper Pinson et al. [2022] presents an analytics market in an online context. They show that a regression market can be constructed using any linear model minimizing a convex loss function as the market model. They also handle non-stationary distributions. However, they assume that the set of features is constant and do not support adding or removing features.

In Han et al. [2022], a regression market using the LASSO regressor is built, which attempts to meet a need of both data buyers and data sellers. Sellers generally want a minimum payoff before they are willing to disclose their feature, which they can decide individually in the LASSO based market. Buyers also do not want to pay for data that is barely informative, which is also avoided with this regularized market. Thus, the market improves results for both buyers and sellers. However, there ends up being a cost in the market for this feature selection, as buyers must pay for decreases in the regularization term.

The paper Cao et al. [2017] uses an iterative auction mechanism where sellers report a privacy cost for selling their data and buyers report a utility score for receiving it, thus allowing direct sale of data, accounting for privacy costs. The paper Acemoglu et al. [2022] shows that highly correlated data can lead to too much data being shared in a marketplace. Every data seller can have an individual privacy cost, but since data is correlated, one seller can reveal information about others. This is a negative externality of data sharing, and drives down the price of data. It can result in the data market reducing social welfare. The paper finds that a mediator that transforms the data by decorrelating it with data owned by privacy-preferring agents before providing it to buyers can improve welfare and bring social welfare higher than the situation with no data market. This mirrors regression markets where the market model increases privacy for feature sellers.

An auction mechanism for data is presented in Agarwal et al. [2023] that only considers the utility of data for buyers and the negative externality unto other buyers from a buyer receiving the data. This results in either all data or no data being allocated to a buyer, based on a simple threshold. If many buyers in competition enter the market, it can result in a market where every buyer pays to avoid a competitor acquiring data, but no buyer receives the data.

Payment allocation between feature owners in analytics markets are based on the Shapley value, which is a concept from cooperative game theory introduced in Shapley [1953].

It is also used in e.g. machine learning explainability to explain the contribution from every input feature to a certain model output, e.g. in Chen et al. [2021].

A central issue within analytics markets is the problem of replication robustness, explained in Agarwal et al. [2019]. The essence of the problem is that under the most common revenue allocation policy, a feature seller can duplicate their own data and sell it twice and thus increase their portion of the total revenue in the market if other correlated features exist in the market. The preprint article Falconer et al. [2024a] investigates an alternative policy where a different causal understanding of Shapley values is used and shows that this revenue allocation policy is robust to replication; however, it also increases financial risk for feature sellers.

While online regression is one option for updating a forecasting mechanism over time, many actors in the electricity markets instead use batch regression models but retrain their model every so often, often every day. This is equivalent to using an online model to perform predictions with a certain forecasting horizon. One alternative in this case to an online model that supports a varying number of features is a batch learning model that supports the same. A framework for this is presented in Wen et al. [2024] that implements a batch learning approach for arbitrary missing features using Fully Conditional Specification (FCS) as originally presented in Van Buuren et al. [2006]. There are many other options for batch learning approaches with missing features, e.g. any type of imputation method. Many such imputation methods are presented in Van Buuren [2018].

Another alternative to imputation is to retrain the model with only the features present at prediction time. In Tawn et al. [2020], this is shown to outperform imputation for wind power forecasting. However, this can be computationally a lot more expensive than imputation, since one can end up having to retrain the model for every single potential set of features. This method also still requires in-sample imputation if features are missing in-sample.

Online machine learning is the field of machine learning models that learn not from a batch of data in one go, but instead from receiving data in a streaming fashion. Online learning is usually based on an iterative update of model parameters using data for one or a few time steps at a time. Online learning also allows for the model to adapt to changes in the underlying data, i.e. a non-stationary distribution. This can be done e.g. using exponential forgetting algorithms like in Pinson et al. [2022].

Several state-of-the-art online learning models that support varying feature spaces are presented in the survey article He et al. [2023], where they name this subfield Utilitarian Online Learning. The subfield is concerned with online supervised learning models that are computationally efficient and can handle arbitrarily varying feature spaces. The models presented are all classifiers, and they categorize the models into three classes. They do not present a regressor.

There is then an open space in the literature both regarding a computationally efficient online regression mechanism for varying feature spaces and regarding a market built on one such mechanism, which would allow us to remove the assumption that no participants enter or leave the data market.

1.5 Contributions

Motivated by this problem space, we would like to develop an online regression market that supports a varying number of participants in the market over time. An analytics market requires a regression mechanism that can work with the dynamics of its dataset, so we will need to investigate regression mechanisms that work when the feature space varies

over time. The regression mechanism would be employed for multi horizon forecasting problems, since this is the nature of renewable forecasting problems in the electricity market.

We will show that a market based on a developed regression mechanism with varying feature space fulfills the same properties for revenue allocation as seen in previous research, covering Shapley fairness, truthfulness, and individual rationality when using the traditional Shapley allocation policy.

Additionally, we will examine methods for allocating revenue in markets with a changing set of participants. We will show that some alternatives that one might consider to the traditional Shapley value revenue allocation policy do not fulfill the above properties, and we will show the consequences of using the Shapley value in varying feature space, along with a new truthfulness property relating to missing features known as *truthfulness under imputation*.

2 Preliminaries

In this chapter, I will introduce the theoretical preliminaries needed to understand the contributions and results of this thesis. It is split into two main sections, one on machine learning, and one on game theory.

2.1 Notation

This section contains notation for use as reference throughout the thesis.

$\hat{y}$ Prediction of a supervised learning method

$l(\hat{y}, y)$ Loss function, takes in the predicted value $\hat{y}$ and the true value y

w Coefficients of a linear model

$L(X, Y)$ The empirical risk, i.e. mean loss over the dataset X, Y

$h(x)$ The logistic function $h(x) = \dfrac{1}{1 + e^{-x}}$

$\Pi_{\mathbb{R}^A}(x)$ The projection operator, projects the vector x onto the space $\mathbb{R}^A$ so that the features or indices A are included, where A is an index set $A \subseteq \mathbb{N}$

f Function representing a relation between features and targets

F Family or space of functions

$\mathcal{X}$ Feature space

$\mathcal{Y}$ Target space

v Characteristic function of a cooperative game

ω Coalition of players in a cooperative game or set of features in a market

N Set of all players in a cooperative game

ψ Revenue allocation or Shapley value

π Revenue for all or some features

γ Willingness to pay function for a loss reduction, willingness to pay coefficent

$\mathcal{L}$ The likelihood function

$\mathcal{F}, \mathcal{G}$ Objective functions

u_t Index set of all features seen up to and including time step t

d_t Index set of all features present at time t

x_t Features reported at time t

$x_{i,t}$ Feature i at time t

τ Step size for gradient descent update

G Reconstruction matrix for linear reconstruction method

z Intermediate calculation based on values of x

ρ Feature correlation or parameter generating feature correlations

T Total number of time steps

ΔT Prediction time horizon, i.e. number of time steps where predictions are made but targets are not available

$\mathcal{N}$ Normal distribution

U Uniform distribution

Σ Covariance matrix

θ True parameters of linear relationship

β_1 OCDS L1 regularization constant

β_2 OCDS graph laplacian regularization constant

α OCDS reconstruction error scaling constant

For clarification about the projection operator: If $A = \{1, 3, 5\}$ and $x = [1, 0, -4, 2, 3]$, then $\Pi_{\mathbb{R}^A}(x) = [x_1, x_3, x_5] = [1, -4, 3]$.

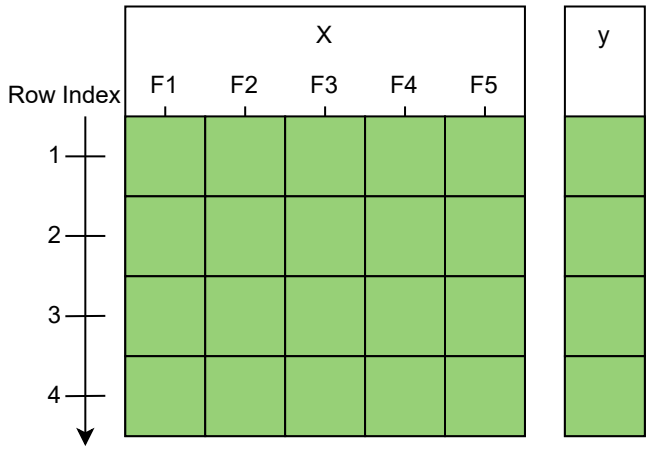


Figure 2.1: A diagram of a tabular dataset. Every green square is a single numerical or categorical value. The dataset has 5 columns and 4 rows, i.e. 5 features and 4 observations. Attached to every row is also a true target, seen in the vector y.

2.2 Machine Learning

Machine learning is the field of constructing algorithms that learn from data. There are several subfields within machine learning. Within machine learning, there is e.g. the supervised and unsupervised distinction. Unsupervised learning attempts to make sense of unannotated data, while supervised learning attempts to learn the relationship between a set of inputs and a true label attached to the inputs.

We are interested in supervised learning problems, since the problems that are sold in analytics markets fall within this umbrella.

2.2.1 Supervised Learning

Within supervised learning, the two most central problems are regression and classification problems. Both problems assign a true label for every observation in a set of observations. In regression, the true label is continuous, while it is a discrete category in classification known as a class. The problems then concern finding a function that predicts the label based on the observation. Regression problems and classification problems share many facets and are quite similar under the hood.

When conceptualizing supervised learning problems, we often see our data as a table, known as a tabular dataset. Here, every row is an observation, and every column is a feature. For every row, a true label or class known as the target might be attached. A tabular dataset is shown in Figure 2.1.

A supervised learning method generates based on every row indexed by t a prediction $\hat{y}_t$ of the true label y_t. It does this based on some function $f : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X}$ is the space that the features exist in or the *feature space*, and $\mathcal{Y}$ is the space that the targets exist in or the *target space*. Then, predictions are made as $\hat{y}_t = f(x_t)$. The problem of supervised learning is the problem of choosing a "good" function f, by some understanding of the word good.

Features and targets can come from different spaces depending on the specific supervised learning problem. Very commonly features are continuous, in which case they come from the real numbers, or a subset of the real numbers, but they can also often be discrete in case of ordinal or cardinal features. Targets can also be continuous or discrete, in many different ways, e.g. $\mathcal{Y} = \{0, 1\}$ for binary classification, $\mathcal{Y} = \{0, 1, 2, \ldots, K\}$ for

K-class classification, or $\mathcal{Y} = \mathbb{R}$ for ordinary least squares regression, to name a few. We also do not always explicitly predict a single numerical value for the target — we could e.g. predict a distribution, in which case the space $\mathcal{Y}$ could be more complex.

Most supervised learning methods choose a function f from a family of functions F, choosing based on a dataset. This dataset is known as the *training* dataset, and choosing a function based on the dataset is known as *fitting* the model. Often, the family of functions F is a space with a set of parameters that can be chosen by choosing the ”best” parameters, by some definition of ”best”. To do this, we need to decide what ”best” means.

2.2.2 Maximum Likelihood Estimation

We can model the relation between the features and targets as a conditional probability distribution, assigning a probability density function $p(y|x)$ to the targets given the features. By finding the parameters of this distribution, we can predict values for new observations by assuming that the distribution of these new data points is the same as the distribution of the data points we used to estimate our distribution parameters. We then naturally get a function f that predicts our regression target for a certain observation, $\hat{y}_t = f(x_t) = \mathbb{E}[y|x_t]$.

To determine the parameters of the distribution, we can use the *likelihood function*. The likelihood function gives the probability of a certain observation given the parameters $\theta \in \Theta$ of the conditional distribution, where Θ is the parameter space of the distribution. It is written

$$\mathcal{L}(\theta; y, x) = p_\theta(y|x) \tag{2.1}$$

where p_θ is the probability density function for the conditional probability distribution with parameters θ. We can estimate the parameters by finding the parameters that maximize the likelihood function, which is known as the maximum likelihood estimate, or MLE. This estimator has some nice properties. It e.g. guarantees convergence as the sample size goes to infinity, known as asymptotic convergence. Using the MLE estimator is a solution for finding a ”good” function to map from features to targets. It also lets us choose what we mean by a ”good” function by our choice of conditional probability distribution. Once we estimate the parameters of the distribution, we can use the expected value as our function f from features to targets. For a more thorough introduction to maximum likelihood estimation, see Sugiyama [2016].

Assuming that every observation x_t, y_t is independent and identically distributed, the likelihood function is the product of the likelihood for every observation individually. This naturally leads us to take the logarithm of the likelihood function.

$$\log\mathcal{L}(\theta; y, x) = \log \prod_t^T \mathcal{L}(\theta|y_t, x_t) = \sum_t^T \log\mathcal{L}(\theta; y_t, x_t) \tag{2.2}$$

Usually, it is more practical to maximize the log likelihood function, as we will see later in Section 2.2.2.

Within machine learning we also have the concept of the *loss function*. The loss function $l : \mathcal{Y}^2 \to \mathbb{R}$ outputs a number known as the *loss* given the predicted label of the model $\hat{y}_t$ and the true label of the model at this time step y_t. The loss function corresponds to the negative log likelihood, sometimes with some constant scaling factor or term which does not change the optimum of the problem. Minimizing the mean loss over the dataset

is also known as the empirical risk minimization problem, since it minimizes the expected loss over the observed data. We prefer for the empirical risk minimization problem to be convex, since convex optimization is a well studied field and methods exist for solving convex optimization problems that guarantee convergence.

Note that in machine learning, the word model refers to 1) a specific function that maps from an observation to a target, it refers to 2) a family of functions from where you can choose a prediction function by solving the empirical risk minimization problem, and it also refers to 3) a broader family of functions that has both parameters that can be decided by solving the empirical risk minimization problem, and more parameters known as *hyperparameters* that must be decided before solving the empirical risk minimization problem.

Supervised Learning Example

To understand the concept of supervised learning, let us work through a hypothetical example of a supervised learning task, specifically the task of our wind farm from earlier forecasting its production forward in time.

Let us say that the wind farm continuously measures their current production as well as the wind speed at their location, and they are interested in predicting their production one hour ahead. The current production measure and the wind speed are then the features of the tabular dataset, and the regression target is the production measure one hour later. Let us say that the wind farm does this forecasting once an hour. The table would then have an observation for every hour of the electricity production and the wind speed during that hour, and after the next hour has passed, the electricity production the following hour will be added as the true label of that observation.

Our goal is then to learn some relation between the features and the target. We do this by deciding on a model to use. This model can be selected based on the data we have. We can choose parameters of the model or input whatever information the model might need from this dataset. This is known as training the model.

Let us say we decided to use a model with some unknown parameters that we can decide using maximum likelihood estimation. To determine the parameters of our model, we decide on a conditional probability distribution for the targets given the features, and minimize the negative log likelihood of the parameters.

We now have a function $f(x) = \mathbb{E}[y|x]$ of the features $x \in \mathcal{X}$ that outputs a prediction. This is what we set out to find; the function f is our trained supervised learning model. We can then take our trained model and use it on current generation and wind speed measures to forecast the production in an hour.

Regression

Regression means estimating the dependence of a continuous variable on a set of other variables, like we saw in the section above. The most well-known regression problem is predicting a linear function f to estimate the regression targets Y based on the input features X.

Linear regression can be understood in terms of Maximum Likelihood Estimation. Specifically, ordinary linear regression assumes a relationship between the features and the regression targets of the following form

$$Y = X\theta + \epsilon \tag{2.3}$$

Where ϵ is independent identically distributed Gaussian noise, $\epsilon \overset{iid}{\sim} \mathcal{N}(0, \sigma^2)$. The conditional probability density function is then

$$p(y_t|x_t) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}(\frac{y_t - \theta^\top x_t}{\sigma})^2} \tag{2.4}$$

which has the parameters θ and σ. The likelihood function can be found from this by taking the product of the conditional probability density function for every observation x_t, y_t in the training set. By taking the logarithm, we then get the log-likelihood function:

$$\text{log}\mathcal{L}(\theta, \sigma^2|Y, X) = \text{log} \prod_t^T \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}(\frac{y_t - \theta^\top x_t}{\sigma})^2} \tag{2.5}$$

$$= \sum_t^T -\frac{1}{2}\text{log}(2\pi\sigma^2) + (-\frac{1}{2\sigma^2}(y_t - \theta^\top x_t)^2) \tag{2.6}$$

It turns out that the maximum likelihood estimator of θ does not depend on σ at all, as it cancels out when setting the gradient equal to zero:

$$\nabla_\theta \text{log}\mathcal{L}(\theta, \sigma^2|Y, X) = \nabla_\theta(-\frac{T}{2}\text{log}(2\pi\sigma^2) - \sum_t^T(-\frac{1}{2\sigma^2}(y_t - \theta^\top x_t)^2)) = 0 \tag{2.7}$$

$$\sum_t^T (y_t - \theta^\top x_t)x_t = 0 \tag{2.8}$$

We see that negating the log-likelihood function gives the mean squared error (MSE) with a factor and a constant term that do not matter for the optimization problem. The mean squared error is a common loss function in machine learning and is effectively the default loss function used for regression problems. The MSE is what arises when you assume that the regression target is distributed by a linear function of the features plus independent identically distributed Gaussian noise. When solving regression problems, we can choose our model f by finding the function minimizing the mean squared error.

Other choices can be made for the conditional probability distribution which result in other loss functions, but we will stick to the one that gives rise to mean squared error for regression in this thesis since it is simple and maps well to many common use cases.

Classification

Regression and classification work largely the same way. They do have a central difference in the structure of the output. In regression, the output is continuous, and in classification, the output is theoretically discrete, though classification methods usually output not just the expected class of an observation, but the probability of an observation belonging to each of the potential classes.

Binary classification problems, meaning problems where we classify observations into only two categories, are usually solved by transforming the output of some regression mechanism through a function that squishes the entire real number line into the interval [0,1], and use this output as the probability of being of class 1. Let us investigate how to solve the binary classification problem. We base our investigation off of Grosse [2019].

We need to determine a loss function for the binary classification problem. Let us think about what we want to optimize for in a classification problem. At its base, we want

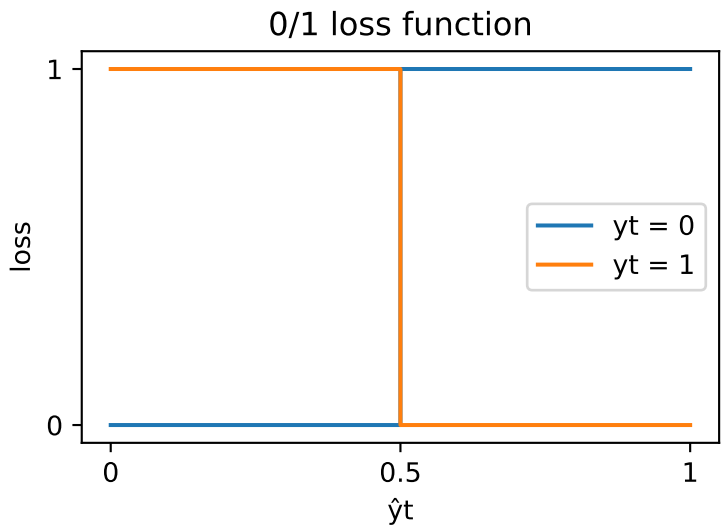


Figure 2.2: The 0/1 loss function, shown for both $y_t = 0$ and $y_t = 1$. It has a clear discontinuity at $\hat{y}_t = 0.5$. We can also see from the shape of the function that it is not convex.

to maximize accuracy, i.e. minimize the difference between predicted classes and true classes. We call this the 0/1 loss.

$$l(\hat{y}_t, y_t) = \begin{cases} 0, & \text{if } \lfloor \hat{y}_t \rceil = y_t \\ 1, & \text{otherwise} \end{cases} \tag{2.9}$$

Where $\lfloor x \rceil$ is the rounding function. The function is shown in Figure 2.2. This function has a derivative of zero everywhere, except for at the point $\hat{y}_t = 0.5$ where the function is discontinuous and does not have a derivative. The derivative is useful for optimization, as we will see later, so optimizing for this loss function is impractical. On top of that, the function is not convex. We then do not want to directly minimize the 0/1 loss. We instead use what is called a *surrogate loss function*, a loss function that represents almost what we want to optimize, but is easier to optimize for. We can find a surrogate loss function by expressing classification in terms of maximum likelihood estimation.

Considering binary classification as a Maximum Likelihood Estimation problem, we need to assume some conditional distribution of the targets given the features. A very sensible choice is to assume that the targets are distributed by a Bernoulli distribution with a certain probability p of being the positive class and $1 - p$ of being the negative class, where p is a function of the features. The likelihood function given a true vector of targets y and a predicted vector of probabilities $p = \hat{y}$ is then

$$\mathcal{L}(\hat{y}; y) = \prod_t^T (\hat{y}_t)^{y_t} (1 - \hat{y}_t)^{(1-y_t)} \tag{2.10}$$

The log-likelihood function then becomes

$$\log \mathcal{L}(\hat{y}; y) = \sum_t^T (y_t \log \hat{y}_t + (1 - y_t) \log (1 - \hat{y}_t)) \tag{2.11}$$

This log-likelihood function when negated is also known as binary cross-entropy (BCE) loss. This is the most common surrogate loss function for binary classification. It is also convex, which is good for solving the empirical risk minimization problem.

We still need to decide how the predicted probability $\hat{y}_t$ depends on the features x_t. A common type of binary linear classifier assumes that there is a linear relation between the odds of being of class 1 and the features. This is known as logistic regression, and the relation between $\hat{y}_t$ and x_t is then given by $\hat{y}_t = h(\theta^\top x_t)$, where $h : \mathbb{R} \to]0, 1[$ is the logistic function $h(x) = \frac{1}{1+e^{-x}}$ that converts odds which span the real numbers to probabilities in the space $[0, 1]$. This then fully specifies the conditional distribution, allowing us to estimate the parameters θ of the distribution by maximizing the log likelihood function.

Gradient Descent
We generally determine parameters for classification and regression models by minimizing the average loss, also known as the empirical risk. In the case of ordinary linear regression, the optimization problem has an analytical solution, so we can find the parameters analytically.

There are many other parametric supervised learning problems, though, and most of them do not have an analytic solution for the weights of the model. For example, the binary linear classifier based on binary cross-entropy loss and the logistic function does not have an analytic solution for a linear relation between the odds of being class 1 and x_t. Then, we need to solve the optimization problem in some other way.

One useful idea for solving this is that for some families of functions F, we can differentiate the empirical risk with regards to the parameters of the function. This is e.g. true for the family of linear functions, where we can find the gradient of the empirical risk given the coefficients of the features as long as we have a differentiable loss function. We can then follow the gradient backwards towards the minimum of the function which is known as gradient descent, as we follow the gradient "downwards", i.e., follow the negative gradient towards the minimum of the function. The concept is illustrated in Figure 2.3. In every iteration of the gradient descent algorithm, the parameters of the model w are updated by following the gradient of the empirical risk $L_w(X, Y)$ backwards with a certain step size τ.

$$w := w - \tau \nabla_w L_w(X, Y) \tag{2.12}$$

The step size can vary at every iteration. If the step size goes to zero and the cumulative step size goes to infinity, $\tau_t \to 0$ as $t \to \infty$ and $\sum_t^T \tau_t \to \infty$ as $T \to \infty$, then gradient descent is guaranteed to converge for a convex optimization problem. Model families F where we can differentiate the loss function in the parameters are known as differentiable models. If the loss function is differentiable or even subdifferentiable, we can perform gradient descent with any differentiable model. If the loss function is convex in the model parameters as well, then we can guarantee that following the gradient descent algorithm will converge to the true minimum of the empirical risk. Then, using gradient descent for determining the weights of a linear model using a convex loss function guarantees converges.

2.2.3 Missing Data & Imputation

In a machine learning problem, data can be missing. If we go back to our motivating example for regression, we could for example have our wind speed sensor break down for an hour and thus have missing data for a certain time step. If we want to use a machine learning model that uses a certain feature as input but the feature is missing for a timestep t, we will have to somehow deal with the fact that the feature is missing. The data could

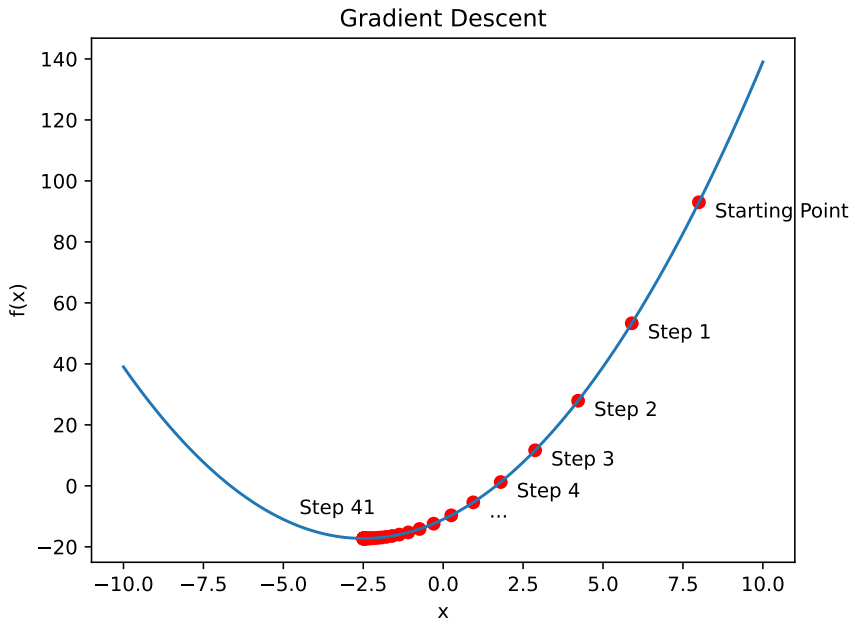


Figure 2.3: Gradient descent. The algorithm is given a starting value for x. It iteratively follows the gradient backwards at that point moving by a certain step size and converges to the minimum of the minimized function. The minimized function is $x^2 + 5x + 11$, which is convex.

	In-sample	Out-of-sample
Impute	Skew training distribution	Produce suboptimal prediction
Discard	Do not learn from observation	Produce no prediction

Table 2.1: Possible solutions to missing data in the in-sample and out-of-sample case. Note that while imputation can skew training distribution for all data, so can discarding data if the missingness is informative, see MNAR data.

be missing from the training dataset, in which case we would either have to remove that row when training our model, or somehow replace the missing feature with another value, known as imputation. It could also be missing when we want to make a prediction on a new observation, in which case we need to replace it with a value to be able to use our model to generate predictions for this observation. The different decisions and their outcomes regarding imputing missing data are illustrated in the Table 2.1. Even if a model that inherently supports missing data is used, that model must internally have some method for dealing with missing data. For an introduction to missing data, see Harrell [2001].

Replacing the missing feature with another value, however it is determined, is known as imputation. Based on why data is missing, imputation methods can be more or less appropriate and introduce more or less bias in the data. Imputation methods are discussed below, and further covered in Longford [2005].

Types of Missing Data
Missing data mechanisms are classified into three categories:

1. MCAR — Missing Completely At Random
2. MAR — Missing At Random

3. MNAR — Missing Not At Random, also known as informative missing (IM)

MCAR data is data that is missing independent of the data itself, so the data loss mechanism has no relation to the data. MAR data is missing dependent on the rest of the data, but independent of the missing data. This could e.g. be seen in our motivating example if high wind speeds make the electricity production measuring apparatus malfunction, thus making production data go missing dependent on the observed wind speed. MNAR data is missing dependent on the missing data, e.g. if very high wind speeds also caused the wind speed meter to malfunction so the value of the feature itself is the reason for the feature going missing.

MCAR and MAR data has the advantage that it can be reconstructed from joint distributions between features with no introduced bias from this reconstruction. It has the disadvantage that the fact that the feature is missing carries no extra information, so missing information is non-informative and only negative for overall model performance.

MNAR data has the issue that it cannot be accurately reconstructed from present features. Any type of reconstruction will introduce a bias since the missing data is from a different distribution than the present data, and even discarding rows where data is missing will introduce bias. However, the fact that data is missing is itself informative and can be used as a feature in the machine learning model.

In this thesis, we assume that missing data is either MCAR or MAR, since these types of data can be imputed from the present data without introducing bias from the reconstruction.

Imputation Methods
As mentioned earlier, whenever data is missing, we have the choice of imputing or discarding it. Imputation can be done based on very simple best guesses based on historical observations of the feature, all the way up to quite complex model-based imputation methods using present features, recent observations, and statistical methods. Some common imputation methods are:

1. Mean/Mode Imputation
2. Using Domain Knowledge
3. Tabular Supervised Learning Models
4. Auto-regressive Imputation

1) Mean/mode imputation replaces the missing value with the historical mean or mode of the feature. This is simple and practical but might considerably skew the distribution. Non-linear supervised learning methods can significantly deteriorate if mean/mode imputation is used in the training set due to the change of the feature distribution.

2) Sometimes, domain knowledge can be used to reconstruct missing data. This is usually in situations with either MAR or MNAR data.

3) Tabular supervised learning models, e.g. k-nearest-neighbors or random forest, can be used for reconstruction, by training on historical data and reconstructing missing data using the trained model and present features. This allows capturing quite complex relations between data, but pushes down the problem of imputation another step, since these regressions will then also need to somehow deal with missing data. One such method is Fully Conditional Specification, see Van Buuren et al. [2006].

4) Sometimes, observations depend on recent observations in an autoregressive relationship. If this is the case, then features can be reconstructed based on former feature values. However, in this thesis we assume that all observations are independent and identically distributed since some analysis becomes a lot more complicated if auto-regressive imputation is a viable option.

2.3 Online Learning

Online learning is a different method of machine learning than the tabular learning approach. In online learning, observations and targets arrive in a streaming fashion, and the model is updated continuously with new information.

Methods for learning model parameters are evaluated differently, since not only the found parameters are important, but also how quickly they are found. If a model e.g. has high loss compared to other methods and only starts performing well after many observations, but then gets better than other methods after going through most of the observations it will see, it might not perform very well for an online learning task. This could e.g. just be a method with slow convergence, which in a batch learning setting would be an inconvenience due to long training times, but in an online setting could materially impact the results of using the method due to the poor results it would provide before converging.

To quantify this evaluation metric in online learning, we introduce the term *regret*. Regret is a term that comes from adversarial game theory. Let us understand it with an example. Imagine that you are playing a game against an adversary where he is secretly deciding on a random number between 0 and 1 every round of the game from a fixed distribution. You then want to decide on some number for your guess on every round. You are then penalized based on the distance between your guess and the random number every round, where the penalty is the loss. Your regret is then the difference between the sum loss you experienced and the sum loss experienced by the best fixed choice you could have made. Seen from a supervised learning perspective, the regret is the difference between the sum loss you experienced and the sum loss you would have experienced if you chose the best fixed parameters.

In online learning situations we want to minimize our regret since sum loss is not well defined for a dataset that can keep growing. We prefer models that have more slowly increasing regret, and generally a good goal is sublinear regret, i.e. regret does not grow linearly over time but instead grows more slowly the more observations have been seen. Sublinear regret in an online setting is similar to guaranteeing convergence in a batch learning setting. A general introduction to online modern learning can be found in Orabona [2023].

Online learning methods usually iteratively update a set of model parameters, sometimes one time step at a time, and sometimes using small batches of data. In this thesis, online learning methods will run one time step t at a time. It will for that time step receive a set of features x_t, and then predict a value $\hat{y}_t$ for the target. Then, it will eventually receive the true value y_t, and use this to update its prediction model.

Let us consider online ordinary linear regression. Two ways of performing online linear regression are 1) Keeping the matrix $X^\top X$ around, updating it every step, to then invert it and use a Newton-Raphson step to update regression coefficients. This converges quickly, but is sensitive to initial conditions and can have numerical issues if the problem is ill conditioned. It also requires knowledge of the Hessian of the objective function. Another way is to 2) update coefficients using what is known as stochastic gradient descent, which

is a modification of gradient descent for online situations. This does not converge as fast as Newton-Raphson, but is more robust to the problem specification.

2.3.1 Stochastic Gradient Descent

Gradient descent for batch regression involves calculating a gradient over an entire dataset. In online learning, we do not have access to the entire dataset at once. We instead receive one observation at a time, so we cannot calculate the gradient of the loss function across the entire dataset at once like we did for batch learning. We want to minimize regret for the model we choose based on these single observations. We can do this using a variant of gradient descent known as stochastic gradient descent.

In online learning we can compute the gradient over one or a few observations at a time and see this gradient as an estimate of the true gradient. We can then update our model parameters based on this estimated gradient. Due to the sampling error inherent in estimating the gradient based on a subset of the total dataset, this algorithm is known as stochastic gradient descent (SGD).

Stochastic gradient descent lets us bound the regret of our model choice to sublinear regret on the order of $O(\sqrt{T})$ if we can bound the variance of the estimated gradient and choose our step sizes well, meaning that it is an appropriate optimization method in an online setting. It is the only method we will use in this thesis for updating model weights. We use Stochastic Gradient Descent instead of Newton-Raphson since large numbers of missing features can often introduce ill-conditioning for Newton-Raphson for the used models, making a Newton-Raphson step less stable.

2.4 Game Theory

Game theory is the study of strategic interactions between rational agents. Put another way, game theory is the study of how agents will interact if every agent is acting to optimize some measure of personal gain. An interaction between rational agents is known as a game.

The first issue in game theory is then to quantify this personal gain. We assign a mathematical value to the preference of agents known as their utility. The agent prefers the outcome with a higher utility to the outcome with a lower utility, and they will act to maximize their utility. The utility often represents the monetary value of an outcome to an agent. Therefore, the utility of an outcome is also often known as the payoff of the outcome.

A game has a set of possible outcomes O. Every agent i in the set of all agents N has a set of strategies they can choose between, known as their strategy space, S_i. The outcome can be represented as the set of chosen strategies of every agent, $O = \{s_1, s_2, \ldots, s_{|N|}\}$, where $s_i \in S_i$, $\forall i \in N$. When we investigate how agents act in a game, we assign them a utility function that maps an outcome to the real numbers $f : O \to \mathbb{R}$. The output is the utility. A note about naming: The words agent, player, and participant will be used interchangeably to refer to the agents in a game. A basic introduction to game theory from an algorithmic perspective is given in Roughgarden [2014].

The players of a game can have private information. In Bayesian games, an agent does not fully know the value of each outcome based on their own private information, and it depends on the private information of the other agents in the game. This makes the utility of an outcome probabilistic before the game is run from the view of a single player, drawn from a probability distribution based on the private information of the other agents.

2.4.1 Cooperative Game Theory

Cooperative game theory is a branch of game theory in which players can make binding agreements externally to the game, allowing players to join forces as groups known as coalitions. A cooperative game is concerned with the payoff for coalitions of players, and can be defined by a characteristic function $v : 2^N \to \mathbb{R}$ from a coalition ω of players to a sum payoff $v(\omega)$. The coalition of all players is known as the grand coalition.

Shapley Values

Cooperative game theory investigates the question of how to fairly allocate the payoff of a cooperative game and forming definitions for the meaning of fairness. One fair payoff allocation is known as the Shapley value, named after economist Lloyd Shapley. It is built on a definition of fairness understood by examining the effect of adding a player to a coalition, and examining how adding the player increases or decreases the sum payoff. Specifically, it guarantees what is known as Shapley fairness, which is the following properties:

1. Budget Balance — The total payoff from the grand coalition is allocated between participants.
2. Symmetry — If two players have the same effect in all coalitions, then they receive the same portion of the total payout. Equivalently, the way we order our players does not matter.
3. Zero Element — If a player has no effect on any coalition, then they do not receive any payoff.
4. Additivity — If we have two cooperative games and add them together, the sum payoff for each player is the sum of their payoff in the first game and their payoff in the second, i.e. how the payoffs are allocated is the same whether we examine the games as two separate games or as a single game.

These are the fairness properties that are guaranteed by the Shapley value. The Shapley value turns out to be the unique way to allocate payoffs that guarantees these properties. It is calculated by examining the effect of adding a player to every possible coalition that it is not already in. The Shapley value ψ_i for player i in the game given by characteristic function $v(\omega)$ is calculated from the following equation:

$$\psi_i = \sum_{\omega \subseteq N \setminus \{i\}} \frac{|\omega|!(|N| - |\omega| - 1)!}{|N|!} [v(\omega \cup \{i\}) - v(\omega) \tag{2.13}$$

Here, ω is a coalition, and N is the set of all players. $N \setminus \{i\}$ is then the set of all players except player i. $|N|$ is the total number of players. The equation calculates for every coalition ω without the player i the difference between the payoff for ω itself and ω with i added in, and multiplies this by a factor based on the number of orderings of the coalition. These values are then summed. The Shapley value can be seen as a function from a characteristic game to the contribution of every participant to the overall payoff of the game. This overview of Shapley values is based on Knight [2015].

The algorithm for calculating the Shapley values of a player is of runtime complexity $O(2^{|N|})$, where $|N|$ is the number of players. This is generally prohibitively slow. Shapley values can also be calculated approximately, through a scheme presented in Agarwal et al. [2019]. This approximate calculation scheme samples coalitions, thus reducing runtime, but does not give exact values. It is of order $O(|N|^2)$ for a certain choice of precision.

However, we will present the results in this paper using the exact Shapley value, knowing that a practical market implementation could use the approximate Shapley value instead.

2.4.2 Truthfulness

Within game theory, we look at the incentivized behavior of the players in a game. We often have a situation where the participants in a game have some private information and a game then requests them to share that private information. Then, we prefer for our players to truthfully report their private information. If they are incentivized to do this, i.e. they expect to receive a higher payoff from truthfully reporting their private information than any other information, our game has a property known as *truthfulness*.

The reason truthfulness is preferred is that it makes it easier to interact with the game for players. When a game is not truthful, players need to make a decision within their strategy space as to what information to report. This decision can quickly become very complicated, and is often dependent on the distribution of the private information of the other players, as well as the number of other players present. Sometimes, solving for the optimal strategic choice is even intractable. A truthful game bypasses all this, since the players just need to report their private information.

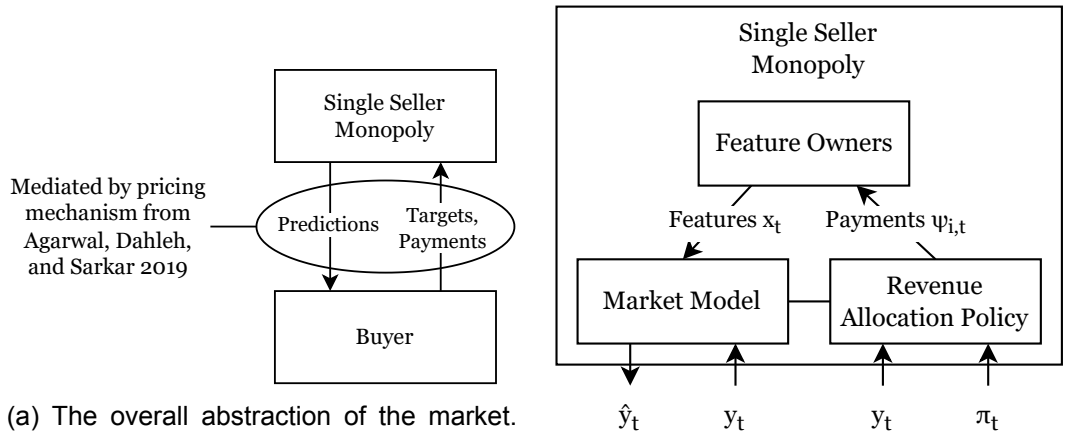
2.4.3 Individual Rationality

The payoff from a cooperative game can be both positive and negative. Sometimes, players have a choice of entering the game or not, in which case we set their utility from not entering the game as zero. Then, when designing markets, we will want our market participants to have a positive utility from entering the market as they will abstain otherwise. When the utility of entering a game is positive, it is known as *individually rational*. In Bayesian games, there are two different versions of individual rationality. Individual rationality in expectation, often just known as individual rationality, means that players expect a positive payoff from entering the game, while ex-post individual rationality (EPIR) means that the players will receive positive payoff from the game in every run of the game, not only in expectation.

2.4.4 Analytics Markets

Analytics markets are markets where participants can sell features and buy predictions for certain machine learning tasks, mediated by a machine learning model. Analytics markets are meant to incentivize exchange of data when data owners and data consumers are in competition in another market, or just not otherwise incentivized to share data. An analytics market can be investigated using game theory to understand the behavior of participants. The market can be seen as a game contains two classes of agents, buyers and sellers, and they interact with the market in different ways. The analytics market was as mentioned earlier formalized in Agarwal et al. [2019]. While we only investigate the seller side of the market, there is already a well-founded solution on the buyer side to incentivize truthful bidding for a buyer in the market, which is presented in the previously mentioned paper.

Though a market can have several buyers, we can clear the market separately for each buyer. We then always consider a market with a single buyer and call that buyer the *central agent*. Every other participant in the market is selling features, and we term it a *supporting agent*. We see the market as an interaction between a single monopolistic seller and a single buyer, and see the seller side internally as a cooperative game between the supporting agents. The cooperative game produces payoff based on how much the features in it improve the predictions of the buyer. The central agent will also themselves bring some features to the table, and this set of features is called ω_c. In terms of players



(a) The overall abstraction of the market. The seller side is seen as a single actor which has a monopoly on selling predictions. Buyers then buy predictions directly from this seller and send back true targets and payments. Buyers also give a willingness-to-pay, mediated by the pricing mechanism mentioned, as well as their own features, the central agent features ω_c.

(b) Internally in the prediction seller monopoly, feature owners sell features to the market in a cooperative game. The model is updated and produces a prediction $\hat{y}_t$, the true targets and payment are reported by the buyer, and revenue is allocated.

Figure 2.4: The overall structure of the analytics market.

in the cooperative game, the coalition ω_c is actually empty, but it is not necessarily empty in terms of features. The structure of the market is shown in Figure 2.4.

The market works by training its model one time step at a time. It receives an observation x_t and the produces a prediction $\hat{y}_t$ that it sends to the buyer. The buyer sends payment and the true value y_t which is used to train the model and allocate revenue. In order to allocate revenue using the Shapley value, the market maintains a model trained on every coalition, and determines the loss using every coalition by examining the coalition prediction and true target, $l_{\omega,t} = l(\hat{y}_{\omega,t}, y_t)$.

When determining the payoff, we only care about reduction in the loss function below what the central agent loss would be with only their own features. We let the payment from the central agent to the market be given by

$$\pi_t = \gamma(l_{\omega_c,t} - l_{\Omega,t}) \tag{2.14}$$

Where ω_c is the features belonging to the central agent, Ω is the grand coalition, and $\gamma(\Delta l)$ is the gain function for the central agent, indicating how much they are willing to pay for a certain decrease in their loss function. The gain function must be monotonically increasing for the buyer-side properties of the market from Agarwal et al. [2019] to hold, but in this thesis we are just going to let it be linear, given by a single positive constant coefficient γ known as the willingness to pay. This willingness to pay is given by a buyer based on their revenue from increased forecasting accuracy in the market. It might be complicated for participants to know their exact willingness to pay, since improved forecasting accuracy often has a complicated relation with exact downstream profit. Future work could investigate methods to learn this willingness to pay over time for buyers in the market.

Similarly, we can define the gain from any single coalition:

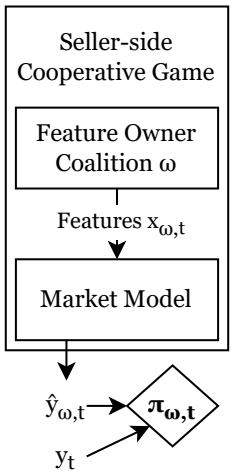


Figure 2.5: The cooperative game that feature sellers are engaged in. Each coalition has a model trained up to time step $t-1$ on that coalition's set of features. Then, the coalition features for time step t come in. They are used to produce a prediction, and the gain $\pi_{\omega,t}$ for the coalition is then given based on the predicted value $\hat{y}_{\omega,t}$ and the true value y_t. All coalition gains are used to calculate feature payments.

$$\pi_{\omega,t} = \gamma(l_{\omega_c,t} - l_{\omega,t}) \tag{2.15}$$

Where ω is a coalition. This mapping is the characteristic function of the cooperative seller-side game that maps a coalition to a payoff, $v(\omega) = \pi_{\omega,t}$. The market uses the Shapley value to allocate the revenue between sellers. We get a sum flow of revenue π_t from the central agent to the supporting agents through the market. The cooperative game that the sellers are engaged in is shown in Figure 2.5, which shows how a single coalition operates and is assigned a certain gain used to calculate shapley values and feature payments.

Market Properties

We want our market to fulfill a number of properties for the incentives of the feature sellers. We want the revenue allocation to be fair according to the Shapley fairness criteria as well as truthful and individually rational in expectation. However we cannot guarantee anything about the out-of-sample distribution. The out-of-sample distribution might drift, which happens regularly in real-world machine learning tasks. Therefore, we instead look at the properties of the market if applied in-sample, i.e. the properties of the market when the model is trained to convergence of the in-sample data and applied on the in-sample data.

At in-sample convergence, the four Shapley criteria come directly from using the Shapley value as the revenue allocation policy. Individual rationality and truthfulness however cannot be guaranteed for an arbitrary regression mechanism. We also need to define exactly what truthfulness means in an analytics market, which is introduced later.

Individual rationality requires that every participant expects non-negative returns. Let us consider a linear model with weights w that minimizes a convex loss function l.

$$\underset{w}{\text{argmin}}\mathbb{E}[l(w^\top x, y)] \tag{2.16}$$

Where x, y is an observation drawn from the in-sample distribution. It is well known that adding an additional feature to this setup will never increase the expected loss since the new weight could always be set to zero, leaving the expected loss unchanged. Therefore, the difference in payoff used to calculate the Shapley value will always be non-negative, so the Shapley value itself will always be non-negative, and thus any seller expects non-negative revenue in-sample and the market is individually rational.

Truthfulness

We want our market to incentivize the sellers to truthfully report their features, but first we must define what truthfulness means in an analytics market. We define truthfulness as the property that any market participant maximizes their expected returns by reporting their features without adding centered gaussian noise. We understand any other transformation as feature engineering, which we allow agents to do freely. This is sensible, since the agents might have features in any representation, and transforming your information should be incentivized if it increases the quality of the market predictions.

In Agarwal et al. [2019], they assume that the prediction model is of a type where adding gaussian noise to input features does not increase prediction accuracy. Pinson et al. [2022] shows that this assumption holds in-expectation in-sample for any linear model that minimizes a convex loss function when the Shapley value is used for revenue allocation and the willingness to pay function is a linear monotonically increasing function in the loss reduction, which is the type of models that we investigate in this thesis.

In-Sample Markets

Other literature on analytics markets present in-sample markets where participants pay for in-sample performance directly. This makes market properties more sensible, since market properties cannot be guaranteed to hold out-of-sample on non-stationary data where the underlying distribution might not be the same as in the in-sample data, but they can be guaranteed in-sample. In some way, paying for in-sample performance makes sense, as it is at prediction time a useful measure for the expected predictive capabilities of your model.

Since participants are not actually interested in the in-sample performance but instead the out-of-sample performance, this thesis will only focus on the out-of-sample market. While we noted before that in-sample performance is a useful measure, it is also not the best measure for performance especially due to overfitting, which is the situation where a model increases its performance on the in-sample dataset, but lowers its performance out-of-sample. For many applications a set of hold-out data will be used to evaluate the model. This data is known as either test or validation data depending on the exact use, and is a better measure of model performance than the in-sample loss. There do exist models and measures where in-sample evaluation is more useful, e.g. regularized models or Bayesian models with coefficient uncertainties, but we do not investigate them in this thesis.

While the properties of the market are only guaranteed in-expectation in-sample, I still argue that leaving out the in-sample market is more appropriate as all users of the market only care about out-of-sample performance. We assume the out-of-sample distribution to be similar to the in-sample distribution. We do though investigate some properties in-sample, since it can show us information about the asymptotic behavior of a market. It might be possible for future research to establish guarantees or bounds on out-of-sample market properties, but this would require making assumptions about the out-of-sample distribution and is out of scope for this thesis.

3 Varying Feature Spaces and Analytics Markets

In this chapter, I will describe the contributions of this thesis to the field of analytics markets and online regression models. I present four main findings:

1. A review of current literature on online machine learning in evolving feature spaces, including the quality of models.
2. A new utilitarian online learning model using linear reconstruction of missing features.
3. A new market property known as truthfulness under imputation.
4. An investigation of the behavior of revenue allocations in varying feature space.

3.1 Evolving Feature Space

So far, we have always thought of our data as having a fixed set of features. We start out expecting that all features are present, and whenever a feature is not present, we think of it as missing and reconstructing it is then known as imputing this feature. In machine learning generally and especially within online learning, it sometimes makes sense to think of feature spaces as evolving over time. Instead of saying that we have a fixed set of features over time, we instead have a set of features $d_t \subseteq \mathbb{N}$ at every time step t. d_t can be different at every time step. The set of all features observed until and including time step t is known as the universal feature space, $u_t \subseteq \mathbb{N}$. Varying feature space is especially sensible when considering features from a group of real-life entities like wind farms, since these can have sensor failures or be decommissioned, or new entities can appear as discussed in the introduction.

The dynamics of feature spaces can be understood by considering the following four events that happen over the course of a feature's evolution.

1. The feature appears for the first time.
2. The feature is observed normally at a time step.
3. The feature is randomly missing at a time step.
4. The feature disappears forever.

Events 1 and 4 only happen one time each, while events 2 and 3 can both happen an arbitrary number of times. This taxonomy explains the dynamics that a model designed for variable feature space must deal with. It is shown as a timeline in Figure 3.1

While this taxonomy is useful, there is an issue with it. From the point of view of the model, there is no difference between a feature that is randomly missing and a feature that has disappeared permanently. In a non-stationary environment, a feature that has been missing for a long time might suddenly appear again but now have a completely different effect in the context of the model. It might then be better to think of a feature that has been gone for long enough as a different feature.

Evolving feature spaces themselves can also be categorized. They are commonly categorized as:

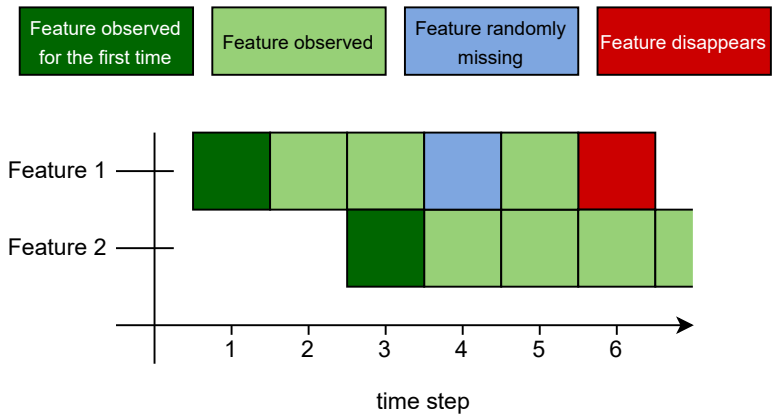


Figure 3.1: The ways that features can evolve over time. The figure shows feature 1 arriving to the model for the first time, then being ordinarily observed. Then feature 2 arrives as well at time step $t = 3$. Then, feature 1 is randomly missing at time step $t = 4$ and comes back again the next time step. Then feature 1 disappears forever at time step $t = 6$.

1. Evolving Feature Streams — There is one set of features for a while, then an overlap with a new set of features, and then the first set of features disappears. Models for this situation are also known as transfer learning models.
2. Trapezoidal Feature Streams — New features appear over time but no features ever disappear, so the feature space grows bigger and bigger.
3. Capricious Data Streams — Some features are randomly missing at many time steps and new features can appear and go missing completely arbitrarily.

These are shown in Figure 3.2, and are useful for understanding the dynamics of evolving feature spaces. Feature space evolution can also be understood by a combination of these; you could e.g. have a feature stream where new features came available over time, and then, once available, would randomly go missing every so often, combining the trapezoidal and capricious types. In this thesis we will limit ourselves to data streams where features all appear at approximately the same time and then go missing randomly, but we do not investigate the dynamics of e.g. completely new features appearing after the model has been running for a long time, or features going missing permanently. This limitation is made simply to limit the scope of the thesis.

3.1.1 Utilitarian Online Learning

The name Utilitarian Online Learning (UOL) refers to online learning methods that natively support evolving feature spaces. The name was introduced by the survey article He et al. [2023]. The goal of utilitarian online learning models is to produce predictions in variable feature spaces in a computationally efficient manner.

The survey article presents six state-of-the-art utilitarian online learning models. It also introduces a taxonomy of current utilitarian online learning models based on their approach to evolving feature space. It splits models into three categories:

1. Passive-Aggressive models
2. Feature Correlation models
3. Evolutionary Ensemble models

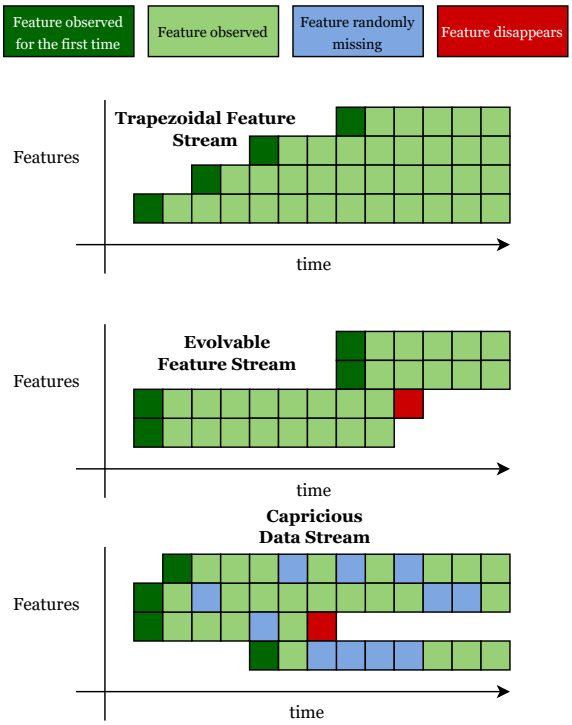


Figure 3.2: Three categories of data streams: Trapezoidal, where new features appear over time; evolvable, where there is an initial set of features, an overlap, and then a new set; and capricious, where features can become available and go missing or disappear completely arbitrarily.

Passive-Aggressive models work by waiting to incorporate new features into their predictions until they have been present for a while. Feature Correlation methods work by reconstructing missing features using some relation between features estimated when they were observed. Evolutionary Ensembles work by evolving a nonlinear ensemble of weak classifiers, typically a forest of decision trees. Two models are presented in every category in the survey article, and all models are classifiers. While they have been tested on some datasets, one issue in this survey article is the relative lack of clarity regarding when to use which of these models.

We will now introduce some models from the literature that we will touch on again later, some in depth and some less so. While we spend some time introducing them here, we will see in the results section that most of the models from the literature do not perform very well. They are also mostly classifiers, and we will need a regressor for our intended applications. The model OLM is partially from literature, but mostly developed for this thesis since the literature that introduced it does not describe its function in any depth. The OLM-S model is a newly constructed model for this thesis.

Online Linear Model (OLM)

The Online Linear Model, or OLM, is a way to implement linear models in a utilitarian online learning context. The model is introduced in the paper He et al. [2019], where it is termed the Online Convex Optimization model. They use it as a binary linear classifier, but they do not present the exact workings of the model since they only use it as a baseline.

An online linear model can be implemented with any conditional probability distribution between the targets and features. We specifically implement two variants, one for regression and one for binary classification. These correspond to the conditional distributions

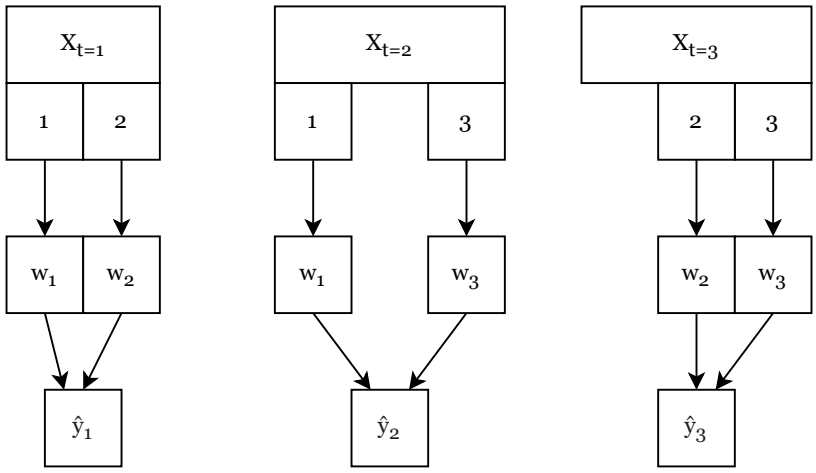


Figure 3.3: Function of the OLM model. We see it generate a prediction every time step based only on the present features. When a new feature enters the model at time step 2, it is initialized with a coefficient of $w_3 = 0$. Only the coefficients for the present features are updated.

specified as y_t normally distributed around $w^\top x_t$, and y_t Bernoulli distributed with probability $h(w^\top x_t)$, respectively. The online linear model handles missing features by mean imputing them. We assume centered input features, which means that mean imputing missing features will be the same as setting them to zero. This will be useful for the implementation of the model.

The OLM has a set of weights w, one for every input feature seen up to time t. This space of all observed features up until time t is known as the universal feature space, u_t. At every time step t, it receives some features x_t. All features are not necessarily present. The set of present features is denoted d_t, so that $x_t \in \mathbb{R}^{d_t}$

The idea of the model is then to receive the features for a time step and produce predictions based on only the present features, zeroing out missing features which is the same as mean imputing them. The weights are then updated. Whenever a new feature appears, it is given a new weight that is initialized to zero. The general function of the model is displayed in Figure 3.3.

For the regression variant the prediction $\hat{y}_t$ is given by

$$\hat{y}_t = \sum_{i \in d_t} w_i x_i + \sum_{i \in (u_t \setminus d_t)} w_i 0 = \bar{w}^\top x_t \tag{3.1}$$

Here, $\bar{w}$ is the vector of weights for present features, $\bar{w} = [w_i \text{ for } i \in d_t]^\top$.

The weights are then updated using stochastic gradient descent. The loss function is the mean square error, meaning that the loss observed at a single time step will be $l(\hat{y}_t, y_t) = (\hat{y}_t - y_t)^2$. The gradient descent update step is then given by following the negative gradient of the loss function with regards to the weights of the model, noting that the gradient is 0 for weights corresponding to missing features

$$\nabla_{\bar{w}} (\bar{w}^\top x_t - y_t)^2 = 2x_t(\bar{w}^\top x_t - y_t) \tag{3.2}$$

This gives us the update step

$$\bar{w} := \bar{w} - \tau 2 x_t(\bar{w}^\top x_t - y_t) \tag{3.3}$$

where τ is a step size parameter.

OLM Classification variant

The classification variant works mostly in the same way. The prediction is formed as

$$\hat{y}_t = h(\sum_{i \in d_t} w_i x_i + \sum_{i \in (u_t \setminus d_t)} w_i 0) = h(\bar{w}^\top x_t) \tag{3.4}$$

where h is the logistic function, $h(x) = \frac{1}{1+e^{-x}}$.

Again, weights are updated using stochastic gradient descent. The loss function for the classification model is the negative log likelihood function from the Bernoulli conditional probability distribution, which is the binary cross-entropy (BCE) loss. The loss is then $l(\hat{y}_t, y_t) = -(y_t \text{log} \hat{y}_t + (1 - y_t)\text{log}(1 - \hat{y}_t))$ for a single time step. The objective function written for a single time step is then

$$\mathcal{F} = -(y_t \text{log} \hat{y}_t + (1 - y_t)\text{log}(1 - \hat{y}_t)) \tag{3.5}$$

Since $\hat{y}_t = h(w^\top x_t)$, we can split the gradient up into a couple of factors using the chain rule. We will define $z_t = w^\top x_t$, and the partial derivative for parameter w_i will then be given by

$$\frac{\partial \mathcal{F}}{\partial w_i} = \frac{\partial \mathcal{F}}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial z_t} \cdot \frac{\partial z_t}{\partial w_i} \tag{3.6}$$

The three factors are found:

$$\frac{\partial \mathcal{F}}{\partial \hat{y}_t} = -(\frac{y_t}{\hat{y}_t} - \frac{1 - y_t}{1 - \hat{y}_t}) = -\frac{y_t - y_t \hat{y}_t + y_t \hat{y}_t - \hat{y}_t}{\hat{y}_t(1 - \hat{y}_t)} = -\frac{y_t - \hat{y}_t}{\hat{y}_t(1 - \hat{y}_t)} \tag{3.7}$$

$$\frac{\partial \hat{y}_t}{\partial z_t} = \frac{\partial}{\partial z_t} \frac{1}{1 + e^{-z_t}} = \frac{e^{-z_t}}{(1 + e^{-z_t})^2} = \frac{1}{1 + e^{-z_t}} \cdot \frac{e^{-z_t}}{1 + e^{-z_t}} = \hat{y}_t(1 - \hat{y}_t) \tag{3.8}$$

$$\frac{\partial z_t}{\partial w_i} = \begin{cases} \frac{\partial}{\partial w_i} \bar{w}^\top x_t = x_i & i \in d_t \\ 0 & \text{otherwise} \end{cases} \tag{3.9}$$

The partial derivative for w_i for $i \in d_t$ is then

$$\frac{\partial \mathcal{F}}{\partial w_i} = -(\frac{y_t - \hat{y}_t}{\hat{y}_t(1 - \hat{y}_t)})(\hat{y}_t(1 - \hat{y}_t))(x_i) = x_i(\hat{y}_t - y_t) \tag{3.10}$$

and the gradient becomes $\nabla_{\bar{w}} \mathcal{F} = x_t(\hat{y}_t - y_t)$. Again, the gradient for any missing feature will be zero, and they can be left out of the update. The update step for the classification version of the online linear model is then given by

$$\bar{w} = \bar{w} - \tau x_t(\hat{y}_t - y_t) \tag{3.11}$$

The model deals with new features by initializing their coefficients to zero. The vector of parameters w can thus grow over time. The pseudocode for the OLM regression variant is shown in Algorithm 1. The runtime complexity for a single time step is of order $O(d_t)$. The memory use is of order $O(u_t)$. The model is a useful baseline model to compare to, since it runs in linear time and is simple.

Algorithm 1 Online Linear Model - Regression

Initialize empty weights vector w
for $t \in \{0, 1, \ldots, T\}$ **do**
  Receive x_t with present feature set d_t.
  Initialize newly appeared features, $w_i := 0 \; \forall i \in \{d_t \setminus u_{t-1}\}$.
  Predict $\hat{y}_t = \bar{w}^\top x_t$
  Update w, $\bar{w} := \bar{w} - \tau \, 2x_t(\hat{y}_t - y_t)$
end for

OCDS

The Online Learning on Capricious Data Streams (OCDS) algorithm is a binary linear classifier from the paper He et al. [2019] and a feature correlation method in the categorization from He et al. [2023]. It reconstructs missing features by averaging the expected value of the feature given each feature in the set of present features.

If e.g. feature j is missing, it is reconstructed as $\tilde{x}_j = \frac{1}{|d_t|} \sum_{i \in d_t} \mathbb{E}[x_j | x_i]$, where tilde is used to refer to reconstructed features. Since it is assumed that features are drawn from a normalized multivariate normal distribution, the expected value of one feature given another is given by the function $\mathbb{E}[x_j | x_i] = \Sigma_{i,j} x_i$ if all other features are marginalized out. These coefficients are stored in a matrix G, $\mathbb{E}[x_j | x_i] = G_{j,i} x_i$. This is the basic idea, where G can then be found by minimizing the reconstruction error.

The reconstruction can be expressed succinctly by forming at every time step the matrix Gr_t, which is the columns d_t of G. The reconstruction is then given by

$$\tilde{x}_t = \Pi_{\mathbb{R}^{(u_t \setminus d_t)}}(\frac{1}{|d_t|} Gr_t x_t) \tag{3.12}$$

Where Π is the projection operator, and projects into the space of missing features in this case. For any index set $A \subseteq \mathbb{N}$ and vector x,

$$\Pi_{\mathbb{R}^A}(x) = [x_i \text{ for } i \in A]^\top \tag{3.13}$$

The matrix G is found by minimizing the mean squared error of the reconstruction. This can only be done on present features since we do not know the values for missing features, so the reconstruction error becomes

$$\|\Pi_{\mathbb{R}^{d_t}}(\frac{1}{|d_t|} Gr_t x_t) - x_t\|_2^2 \tag{3.14}$$

The paper introduces a term based on the supervised loss as well in the objective function for determining G.

The model ensembles predictions based on reconstructed features and present features together. It does this by using a parameter p that it sets based on the cumulative loss of using only present and only reconstructed features. The prediction is given by

$$\hat{y} = p\bar{w}^\top x_t + (1-p)\tilde{w}^\top \tilde{x}_t \tag{3.15}$$

Where $\bar{w}$ is the subset of w corresponding to the present features, and $\tilde{w}$ is the subset of w corresponding to the reconstructed features. The predicted label is then sign$(\hat{y})$, where the positive class is given the value 1 and the negative class is given the value -1. Note that the size of the present and reconstructed subsets can change at every time step, e.g. if all features are present there will be no reconstructed features, in which case the prediction is just $\hat{y} = p\bar{w}^\top x_t$. The weights are then updated using stochastic gradient descent and the mean square error.

The OCDS algorithm also introduces three regularization constants. One is called α, and is used to scale the reconstruction error compared to supervised loss. The two others are β_1, which is an $L1$ loss on the coefficients, and β_2, which sees G as a graph and finds the graph laplacian, and uses this to make coefficients for highly correlated features more similar. The optimization problem solved by the OCDS model is then

$$\underset{w,G}{\text{argmin}} \sum_t^T ((\hat{y}_t - y_t)^2 + \alpha \|\Pi_{\mathbb{R}^{d_t}}(\frac{1}{|d_t|} G r_t x_t) - x_t\|_2^2 + \beta_1 \|w\|_1 + \beta_2 Tr(w^\top L w)) \tag{3.16}$$

Where L is the laplacian of G seen as the incidence matrix for a weighted graph, and Tr is the trace. This problem is then solved by stochastic gradient descent one observation at a time, similar to the OLM. It is biconvex, so convergence is guaranteed by a proper choice of step size, e.g. the step size $\frac{1}{\sqrt{t}}$ used in the original paper.

The OCDS model makes some uncommon decisions in its design. The OCDS model is discussed further in Section 4.2.4.

We also introduce a regression variant of the OCDS model. Since the OCDS model already uses the mean squared error for its loss and outputs a continuous value $\hat{y}_t$ for every prediction, one needs only skip the step of turning $\hat{y}_t$ into a class label and it becomes a regression model.

OLVF

The Online Learning from Varying Feature Spaces (OLVF) algorithm is also a linear classifier from the paper Beyazit et al. [2019], classified as a passive-aggressive learner in the taxonomy from He et al. [2023]. It deals with missing features by projecting observed features into a subspace known as the classifier subspace. The model minimizes a hingle loss with slack which turns out to have some complications regarding markets as will be discussed later.

OVFM

The Online Learning in Variable Feature Spaces with Mixed Data (OVFM) algorithm from the paper He et al. [2021] is also a binary linear classifier. It is a development of OCDS and works generally the same way. The main difference is that it uses Gaussian copula to map features into a latent space and performs classification in this space. This allows for better gradients for ordinal features.

Online Linear Model with Subregressions (OLM-S)

The Online Linear Model with Subregressions (OLM-S) algorithm is a new model developed for this thesis. The model is developed based on the idea of reconstructing missing features using the regression variant of the OLM and using a linear classifier or regressor on the reconstructed and present features.

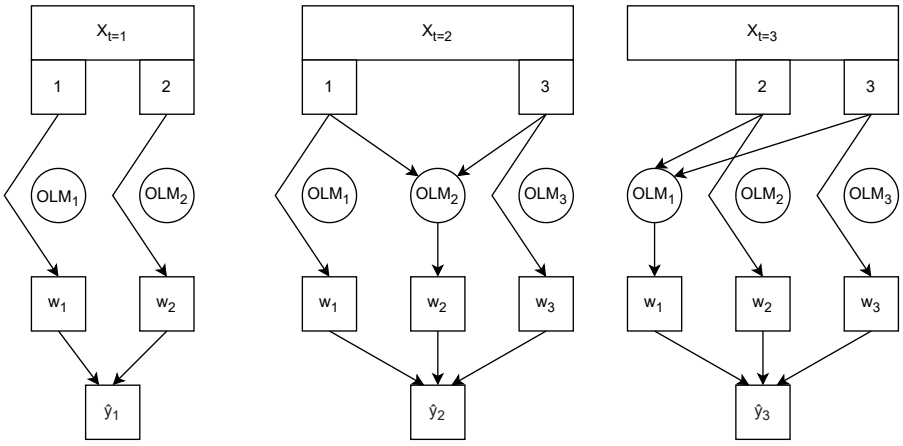


Figure 3.4: The time step by time step workings of the OLM-S model. Seen is the predictive flow of the model. Subregressions OLM_1, OLM_2, and OLM_3 are trained whenever the feature they reconstruct is present. Present features are used to reconstruct missing features. The function of OLM_1 with the same dataset as shown here is shown in Figure 3.5.

The model is a linear model like the OLM that uses stochastic gradient descent for the coefficient update step. The new part of this model is the imputation method. It maintains an OLM regression model for every feature in the model trained to predict said feature based on the remaining features. When features are missing, it then imputes each missing feature using its OLM regression model. An overview of its function is shown in Figure 3.4, and the function of a subregression for the model is shown in Figure 3.5

The simplest way to understand OLM-S is in terms of these sub OLM models used to reconstruct missing features, but it can also represented by letting a matrix G contain the weights for the OLM models. The reconstructed features are then $\tilde{x}_t = \Pi_{\mathbb{R}(u_t \setminus d_t)}(Gr_t x_t)$. Then, the optimization problem solved by the model can be expressed succinctly as follows:

$$w^* = \underset{w}{\text{argmin}} \sum_{t}^{T} (l(\bar{w}^\top x_t + \tilde{w}^\top \tilde{x}_t, y_t)) \tag{3.17}$$

$$G^* = \underset{G}{\text{argmin}} \sum_{t}^{T} \|\Pi_{\mathbb{R}^{d_t}}(Gr_t x_t) - x_t\|_2^2 \tag{3.18}$$

$$\text{s.t.} \quad G_{i,i} = 0 \ \forall i \in u_T \tag{3.19}$$

Where u_T is the set of all features observed over all timesteps. Both of these optimization problems are convex so we can guarantee convergence using stochastic gradient descent with a certain choice of step size. The two optimization problems are called $\mathcal{F}$ and $\mathcal{G}$, respectively. Note that the constraint in $\mathcal{G}$ is ensured by fixing $G_{i,i}$ at 0 when performing the update step. The step using stochastic gradient descent becomes

$$w = w - \tau_1 \nabla_w \mathcal{F} \quad G = G - \tau_2 \nabla_G \mathcal{G} \tag{3.20}$$

where τ_1 and τ_2 are two fixed step size hyperparameters.

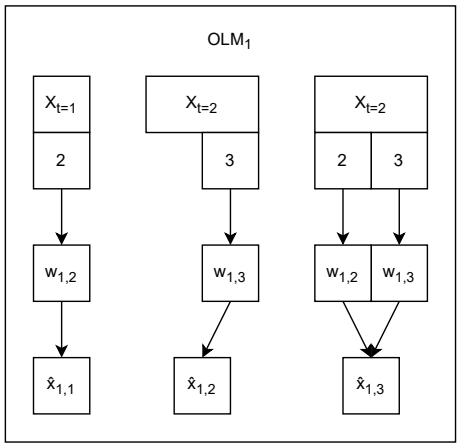


Figure 3.5: The step by step of the subregression OLM_1 reconstructing feature 1 in Figure 3.4. The output from this subregression is only used in the main model at time step $t = 3$, since this is the only point where feature 1 is missing. At the other time steps, the subregression coefficients are trained using other present features, and feature 1 as the target.

The loss can either be specified as $l(\bar{w}^\top x_t + \tilde{w}^\top \tilde{x}_t, y_t)$ for the regression variant or as $l(h(\bar{w}^\top x_t + \tilde{w}^\top \tilde{x}_t), y_t)$ for the classification variant using binary cross-entropy loss. The regression prediction is $\hat{y}_t = \bar{w}^\top x_t + \tilde{w}^\top \tilde{x}_t$, and when classifying, the prediction is instead $\hat{y}_t = h(\bar{w}^\top x_t + \tilde{w}^\top \tilde{x}_t)$.

One advantage of the OLM-S model is that the reconstruction can be trained independently of the true regression targets. This means that in time-ahead prediction situations, the reconstruction can be trained for every time step up to prediction time, and sometimes even later if features are available further ahead in time. This increases the total amount of data that the reconstruction mechanism can learn from.

The prediction step of the algorithm is laid out in Algorithm 2, the update of the predictor or base model is in Algorithm 3, and the update of the reconstruction is in Algorithm 4. Note that in the reconstruction, the matrix G is updated by inserting the columns of the updated Gr_t into the columns d_t of G.

Algorithm 2 OLM-S Single Timestep Prediction

Receive x_t with present feature set d_t
Retrieve Gr_t as columns d_t of G
Reconstruct missing features, $\tilde{x}_t = \Pi_{\mathbb{R}(u_t \setminus d_t)} Gr_t x_t$
Predict $\hat{y} = \bar{w}^\top x_t + \tilde{w}^\top \tilde{x}_t$

Algorithm 3 OLM-S Main Predictor Update

Retrieve x_t with present feature set d_t
Retrieve $\hat{y}_t$ from prediction algorithm
Update $w := w - \tau_1 \nabla_w l(\hat{y}_t, y_t)$

3.2 Market Model Candidates

Based on our preliminaries, we can establish our requirements for a model for our market with varying feature space. We need:

Algorithm 4 OLM-S Reconstruction Update

Retrieve x_t with present feature set d_t
Find $\nabla_{Gr}\mathcal{G} = 2(\Pi_{\mathbb{R}^{d_t}}(Gr_t x_t) - x_t)x_t^\top$, fixing coefficients $G_{i,i}$ to zero.
Update $Gr_t := Gr_t - \tau_2 \nabla_{Gr}\mathcal{G}$
update G using Gr_t

1. A regression model.
2. That can be updated in an online fashion.
3. And deals with varying feature spaces.
4. And is linear in the input features & minimizing a convex loss function.

We will now go through the presented models and investigate how they would fit into a regression market as a market model.

3.2.1 OLM

Looking over the OLM model, we see that it is a regression model that can be updated in an online fashion, works with varying feature spaces, and is linear in the input features. It also minimizes a convex loss function. It is then a valid choice for a market model.

3.2.2 OLM-S

In this section, we will show that the OLM-S model is a valid candidate for a market model. The OLM-S is clearly a regression model that updates in an online fashion, allowing varying feature spaces. It also reconstructs these missing features, which we expect to improve its performance when feature spaces change. Let us look at the representation of OLM-S shown in the optimization problem in Equation 3.17 again and see if we can also fulfill the fourth requirement of linearity. The objective function for the minimization problem is

$$\mathcal{F} = \sum_t^T (l(\bar{w}^\top x_t + \tilde{w}^\top \tilde{x}_t, y_t)) \tag{3.21}$$

$$= \sum_t^T (l(w^\top z_t, y_t)) \quad \text{where } z_{i,t} = \begin{cases} x_i & i \in d_t \\ \tilde{x}_i & i \in u_t \setminus d_t \end{cases} \tag{3.22}$$

Since $z_t = [z_{1,t}, z_{2,t}, \ldots, z_{u_t,t}]$ results from a linear transformation of the input features x_t at every time step t, as $\tilde{x}_t$ is just a linear transformation of the input features x_t, the problem is linear in the input features. We have also assumed that the loss function is convex, so our model requirements are fulfilled, and choosing OLM-S as the market model will give rise to our desired market properties.

3.2.3 OCDS

The OCDS model is online and deals with varying feature spaces. The model can readily be used as a regression model, as mentioned earlier. Similarly to the OLM-S model, it ends up using a linear combination of the input features to produce its predictions, so it is linear in the inputs for the same reasons. It uses the mean squared error, which is a convex loss function.

Then, OCDS is a valid candidate for a market model as well. Note that we do not look at the OVFM model in terms of market properties, but it is a further development of OCDS.

3.2.4 OLVF

The OLVF cannot easily have its loss function changed, unilke the OCDS, OLM-S, and OLM models. The loss function is a hinge loss with a slack variable. Buyers in the market would then need to pay for a reduction in this loss function if the OLVF was employed as the market model. Minimizing a hinge loss with a slack variable is not a loss function that immediately translates to common real-life problems, so buyers would not want to pay for it directly. Therefore, it is not an obvious choice for a market model.

3.3 Revenue Allocation for Missing Features

We know that we can get a fair allocation using the Shapley value. However, within the context of variable feature space, the straightforward Shapley value allocation will also have us calculate Shapley values for the missing features. We sometimes expect negative Shapley values for missing features as will be shown in the results section, so sellers in the market can be forced to pay an amount back to the market in the situation that they do not have data available, and participation can thus carry future risk. This is not true asymptotically in-sample though, and only in cases when small sample sizes or nonstationarity cause poor out-of-sample estimates.

Features can also have an expected positive Shapley value when not present. This might not seem fair for other feature owners. I do show in the results in Section 4.3.3 that out-of-sample, missing features generally have near-zero or negative payments, though. Positive Shapley values for missing features would especially impact an in-sample based market, where an extra feature at one time step would allow the model to overfit on another time step, thus making the feature increase performance at both time steps and receive positive compensation for both. In out-of-sample markets, positive Shapley values can at most arise from a regularizing effect, since the model does not have a chance to overfit on observations it has not yet seen. It could still be argued that the positive Shapley value for a missing feature should not go to the missing feature, but instead be pooled among the present features, since the result could just as easily have been obtained with regularization.

We investigate two alternative allocation policies to more fairly handle missing features. However, both turn out not to work. The market setup in this thesis requires the ordinary Shapley value to produce a revenue allocation method that meets our desired market properties.

Note that while we always call the monetary transaction between the market and feature owner a *payment*, the payment can be positive, in which case it goes from the market to the feature owner, or negative, in which case it goes from the feature owner to the market. If all payments were non-negative we would have an ex-post individually rational market, but we cannot guarantee this.

3.3.1 Alternative Allocation Policy — Redistributing Shapley Values

When reconstructing a missing feature, one revenue allocation policy could be to redistribute Shapley values for missing features from the main model to the features used to reconstruct a missing feature. With this approach, we might e.g. have three features in our overall model. At time step t, feature 1 is missing. It gets assigned some Shapley value ψ_1. In the subregression used to reconstruct feature 1, feature 2 and 3 then get Shapley values $\psi_{1,2}$ & $\psi_{1,3}$. We would then split out the Shapley value in the overall model from feature 1 onto features 2 and 3 based on their Shapley value in the subregression. See Figure 3.6 for a diagram of the method.

A problem is that the subregression Shapley value is not possible to find, since we do not

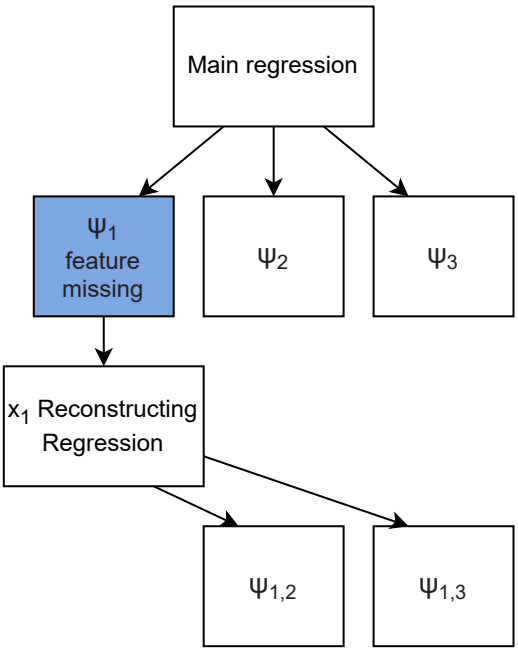


Figure 3.6: Redistributing shapley values for missing features. Here, features 1, 2, and 3 receive a Shapley value in the main regression. Feature 1 is missing at this time step, so its payoff is redistributed between feature 2 and 3 based on their Shapley values in the regression that reconstructs feature 1.

know the true value of the missing feature. We can keep a historical average or moving average Shapley value to serve as an expected Shapley value, but there is no direct way to observe the true Shapley value when reconstructing a feature.

Another proposed option is to expect that features are not missing forever but simply late, in which case the Shapley value would be possible to find once the true value of the missing feature arrives. However, this is a different situation than this thesis seeks to solve. When features are assumed to arrive late, a feature owner cannot have a broken sensor, only issues in transmitting the feature on time. Feature owners also cannot exit the market. One would also need to find a way to incentivize feature owners to report their late features truthfully. All of this is outside of the scope of this thesis.

3.3.2 Alternative Allocation Policy — Ignoring Missing Features

Another possibility is to calculate Shapley values based only on present features. In this allocation policy, we look at which features are present at every time step and calculate Shapley values as if these were the only features in the market. It is illustrated in Figure 3.7. Sometimes missing features are expected to have positive Shapley values, though, in which case feature owners will be missing out on returns and might turn to mean imputation, negatively impacting the market.

This allocation policy also has an attack vector. The policy allows actors to damage the market by providing features correlated with other market features and then letting them go missing later, deteriorating market performance without paying for this reduction in performance. Thus, market owners would not be sufficiently incentivized to keep their features present in the market.

This allocation policy can be used if retraining is used in place of imputation, in which case all missing features will have a Shapley value of zero. This is beyond the work of

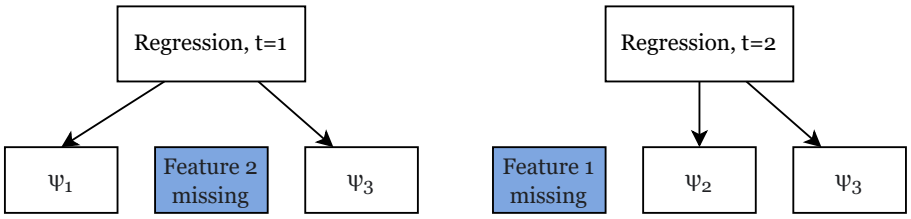


Figure 3.7: Illustration of the alternate allocation policy where missing features are ignored when calculating Shapley values. At $t = 1$, feature 1 and 3 are present and feature 2 is not, so coalitions containing feature 2 are ignored when calculating Shapley values, and it receives no payment. Similarly, at time step $t = 2$, feature 1 is missing and coalitions containing feature 1 are not included when calculating Shapley values.

this thesis, but is discussed in the further works, in Section 5.1.4.

3.3.3 Truthfulness Under Imputation

Previously, we defined truthfulness as incentivizing market participants to report their true features to the market as opposed to adding Gaussian noise to their feature. When features are missing, we have expanded the strategy space by allowing market participants to report their feature as missing and letting the model used in the market handle the missing feature. This introduces a new challenge. Sellers with a missing feature now have the choice between truthfully reporting their feature as missing or imputing their missing feature using historical information, i.e. mean imputation. We assume that mean imputation is the only choice available to feature users; they could of course also provide a regression- or domain knowledge-based imputation themselves, but this complicates analysis significantly and is left for future work.

Mean imputation can have different effects on the market. In some markets, sellers are incentivized to truthfully report their features as missing when they are missing. We term this *truthfulness under imputation*. One reason this is preferable is that in some cases, determining whether one should report a missing feature or mean impute is a difficult problem — in essence, it introduces strategic bidding for the feature sellers, making their interaction in the market more difficult to optimize.

Note that in this thesis we also are making the assumption that observations are independent of previous observations. This is a fairly strong assumption, and doesn't often hold in practice. Without this assumption, another new method of imputation becomes available to market participants, namely autoregressive imputation. Feature owners can supply a value imputed based on their last observed feature value(s) when their feature is missing. Whether a method is truthful under autoregressive imputation will depend on the specific relationship between a feature and its previous values as well as the mechanisms of the feature going missing, and will not be investigated in this work.

Truthfulness Under Imputation for OLM

The OLM method handles missing features by mean imputing. Therefore, the output of the model both at the time step where a feature is missing and every following time step will be the same whether a participant mean imputes or not, and so feature payments will be unchanged. Thus, OLM is actually not strictly truthful under imputation, since participants are not incentivized to truthfully report their features as missing, but suffers none of the consequences of not being truthful under imputation. The model could be called weakly truthful under imputation, or indifferent under imputation.

Truthfulness Under Imputation for OLM-S

Truthfulness under imputation is not guaranteed for OLM-S in this thesis. It is likely possible to show truthfulness under imputation in-sample in-expectation for OLM-S when the distribution of missing features is stationary, but this is left for future work. Out-of-sample, or when the distribution of missing features is non-stationary, the property becomes highly context-dependent.

Practical results about truthfulness under imputation will be discussed in Section 4.3.5.

4 Results and Discussion

This chapter reports on the results of the thesis. We will first investigate utilitarian online learning models divorced from their market context, since this is relevant when we determine which model to use for our regression market. Then, we will investigate market results with varying feature spaces, showing the outcome of the market with both synthetic and real-world datasets.

Note that whenever we test a model, we also add in a constant feature that is always 1 along with the other features which acts like the intercept in an ordinary linear regression and is always present.

4.1 Utilitarian Online Learning

First, we want to investigate which model to use for our market setup. We want a performant utilitarian online learning model, so we investigate and compare available models. In the end we want a regression model as demonstrated in the motivating example — forecasting in the electricity market is most often a regression problem. However, since current research is within classifiers, we investigate these first.

4.1.1 OCDS

OCDS, if converted to a regression model, meets our requirements for a market model. Therefore, we are interested in whether it could be a performant model. We test the OCDS method against the baseline OLM method in a setup similar to what is used in the OCDS paper He et al. [2019]. In this test, we examine both OCDS and OLM as classification methods by comparing their mean accuracy. The test is not very representative of a common use case for online models, but we use it for comparability.

We test on datasets from the UCI databank, since these are the datasets used by most papers within the UOL subfield to argue for the utility of their models. The chosen datasets are shown in Table 4.1, and are from the UCI databank. [Markelle Kelly] We choose these since many models report results for them and they span a substantial amount of the types of datasets these models test on in terms of numbers of features, numbers of observations, and amounts of categorical features.

To prepare the datasets for use, we convert categorical features into discrete numerical features. They are given integer values based on the order that they are observed in. This means that non-binary categorical features are handled in a non-obvious way. This is done since one-hot encoding was found to deteriorate performance.

The test takes in a dataset of features and associated targets X, Y. The time order of these observations is shuffled so that there is no autoregressive relation between observations. Then, for every time step, half of the features are randomly removed, i.e. every time step is missing 50% of features, so that methods must handle missing features. While this does not match the situation where an online model would generally be used, since online models are more useful when the coefficients in the model are changing over time i.e. the distribution is non-stationary, this is the test performed by most current literature in UOL, so we use this test for comparability.

The model first predicts a label for a single time step and then trains on that time step. The test pseudocode is displayed in Algorithm 5. The entire test is run several times,

Dataset	Observations	Features	Non-Binary Categorical Features	UCI ID
australian	690	14	4	143
ionosphere	351	34	0	52
german	1000	20	11	144
wdbc	569	30	0	17
wbc	699	9	0	15
magic04	19020	10	0	159
kr-vs-kp	3196	35	1	22
credit-a	690	15	5	27

Table 4.1: UCI datasets used for classification tests along with the number of observations in the datasets and the number of features per observation. Also provided is the number of features that are categorical and not binary, since we do not process them in a completely standard way. All datasets are binary classification tasks. The ID in the UCI system is supplied for easy lookup.

the mean accuracy is reported, and 95% confidence intervals are constructed using the standard deviation of the mean accuracy.

Algorithm 5 Pseudocode for testing UOL methods

```
Permute X, Y
Initialize model m
for t ∈ {0, 1, ..., T} do
    Create prediction ŷ_t = m.predict(x_t)
    Train model m.train(x_t, y_t)
end for
evaluate model based on predictions ŷ
```

We test OCDS and OLM using this approach. We want to investigate a feasible upper bound for predictive accuracy, so we also use a logistic regression on the same dataset. Logistic regression is the batch learning variant of a linear classifier using binary cross-entropy loss. We let this logistic regression train on the entire dataset with no missing data, and then predict on the entire dataset as well. Its predictions are then in-sample with full information, meaning that we expect it to maximize accuracy within the sample for a linear classifier. So no other linear classification method, like e.g. OCDS, should be able to surpass the logistic regression mean accuracy.

OLM has a hyperparameter in form of a fixed step size. We find this step size by performing a grid search in the set $[1, 0.1, 0.01, 0.001]$ on a sample run through the dataset and picking the step size with highest mean accuracy before running the tests. We use a fixed step size for simplicity.

The original paper performed hyperparameter optimization by grid search for the OCDS regularization constants, which we also perform. OCDS has three regularization constants, α, β_1, and β_2. We search in the space $\alpha \in [1, 0.1, 0.01]$, $\beta_1 \in [0.1, 0.01, 0.001, 0]$, and $\beta_2 \in [0.1, 0.01, 0.001, 0]$. The original OCDS implementation uses the step size $\tau = \frac{1}{\sqrt{t}}$ but this was found to have numerical issues, so we use $\tau = \frac{k}{\sqrt{t}}$, where we perform a hyperparameter search to find k in the space $k \in [1, 0.1, 0.01, 0.001]$. This hyperparameter search is also performed on a sample run through the dataset.

Note that the test in the original OCDS paper is slightly different from the test we perform

	OCDS paper	OCDS Reimplementation	OLM	Logistic UB
australian	0.886 ± 0.011	0.753 ± 0.003	0.767 ± 0.002	0.858
ionosphere	0.906 ± 0.014	0.748 ± 0.001	0.795 ± 0.002	0.937
german	0.827 ± 0.021	0.701 ± 0.001	0.705 ± 0.002	0.755
wdbc	0.951 ± 0.009	0.921 ± 0.002	0.938 ± 0.001	0.993
wbc	0.939 ± 0.005	0.934 ± 0.001	0.938 ± 0.001	0.961
magic04	0.886 ± 0.013	0.732 ± 0.001	0.732 ± 0.001	0.791
kr-vs-kp	0.912 ± 0.006	0.733 ± 0.001	0.750 ± 0.001	0.960
credit-a	0.926 ± 0.014	0.766 ± 0.003	0.783 ± 0.002	0.864

Table 4.2: Mean accuracy. OCDS, both results from the paper and my own implementation, along with the logistic regression upper bound and the baseline OLM model. Note that the logistic regression presents in-sample mean accuracy, while the other models present out-of-sample accuracy. This is done since the logistic regression is meant to serve as an upper bound on possible out-of-sample accuracy.

here. Instead of removing half the features at every time step, they remove up to half of the features where the number of features removed is drawn from a uniform distribution. This means that the original OCDS paper gives slightly more information to the model, since more features are present at every time step. However, due to the choices made in designing the OCDS model, it should perform better when the number of missing features is similar at every time step, i.e. the setup that we use. The OCDS paper does not clarify how they deal with categorical variables in their test.

The results for this test are in Table 4.2. It contains both the results from the original OCDS paper and from the reimplementation of OCDS in this thesis. Note that the logistic regression can feasibly be understood as an upper bound on prediction accuracy since it reports in-sample accuracy for the function type, but the results reported in the OCDS paper regularly break the bound, even though they report out-of-sample accuracy. This is not purely due to a different handling of categorical variables, since it also happens in the magic04 dataset that contains no categorical variables. Therefore, I suspect that the reported results for OCDS are erroneous. My reimplementation is worse than the baseline OLM implementation, so I expect that OCDS is not a performant model for classification.

4.1.2 Other Utilitarian Online Learning Methods

The results lead us to compare to other utilitarian online learning methods presented in the survey article since OCDS is not very performant. We use the same testing method as described above in Algorithm 5.

The added methods are the OLVF classifier and the OVFM classifier. The test presented in the OLVF paper corresponds to the test that we perform, while the test in the OVFM paper is equivalent to the test in the OCDS paper described before. The OVFM model natively accepts categorical variables, so these are not transformed in their test. The tests are then quite similar, with the OVFM paper receiving slightly more information than the rest of the models. We also include the classification version of the OLM-S model developed in this thesis. The two step size hyperparameters of the OLM-S model are chosen by a similar hyperparameter grid search as the step size for OLM.

The results are in Table 4.3

The results for the new methods do not break the logistic upper bound, but no model has consistently higher accuracy than the baseline OLM model. The OVFM model surpasses

	From papers		From this thesis		
	OLVF	OVFM	OLM	OLM-S	OCDS
australian	N/A	**0.783 ± 0.001**	0.767 ± 0.002	0.768 ± 0.003	0.753 ± 0.003
ionosphere	0.774 ± 0.021	0.752 ± 0.001	**0.795 ± 0.002**	0.787 ± 0.002	0.748 ± 0.001
german	0.649 ± 0.008	0.681 ± 0.001	0.705 ± 0.002	**0.709 ± 0.001**	0.701 ± 0.001
wdbc	0.903 ± 0.009	0.918 ± 0.001	**0.938 ± 0.001**	0.935 ± 0.001	0.921 ± 0.002
wbc	0.913 ± 0.008	0.922 ± 0.001	0.938 ± 0.001	**0.941 ± 0.002**	0.934 ± 0.001
magic04	0.644 ± 0.001	**0.762 ± 0.001**	0.732 ± 0.001	0.743 ± 0.001	0.733 ± 0.001
kr-vs-kp	N/A	0.687 ± 0.001	0.750 ± 0.001	**0.753 ± 0.001**	0.733 ± 0.001
credit-a	N/A	0.760 ± 0.001	**0.783 ± 0.001**	0.780 ± 0.002	0.766 ± 0.003

Table 4.3: Mean accuracy. Comparing results from the OLVF and OVFM paper with results that come from the thesis implementation of the OLM baseline and OCDS, along with the new OLM-S model. The best performing method for each dataset is in bold. Note that even though OLM-S performs best on three occasions, it is very close to the performance of OLM.

the OLM baseline in the australian and magic04 datasets, but apart from these, it performs worse that the OLM method. The OLM-S model performs almost identically to OLM in this test. If $\tau_2 = 0$, OLM-S reduces to OLM, so OLM-S can always perform at least as well as OLM for a certain choice of hyperparameters.

The australian dataset has some categorical non-binary features, which is likely to explain why OVFM performs better in this case as it is directed towards categorical features. This does not explain the performance in the magic04 dataset though, where all features are continuous. Even though OVFM does occasionally outperform the baseline OLM, the OVFM paper does not provide information on when this might happen.

Based on these tests, the advice in a classification setting would be to use the baseline OLM model if one would want an online linear classification model that supports variable feature spaces.

4.2 Regression

Since we are looking for a regression model, we move on to investigating online models for regression instead of classification. We now have some initial information on the quality of different UOL methods. The UOL models from literature have not shown very promising results, and we will not spend more time on OLVF and OVFM.

First we examine the time-ahead forecasting performance for OLM, OLM-S, and the regression variant of OCDS.

Our initial regression tests use synthetic datasets where features are drawn from a multivariate Gaussian distribution and targets are given as a linear combination of these features. The distribution has $|N|$ features and a covariance matrix Σ, and a coefficient θ_i is attached to each feature, where the vector of all coefficients is known as θ. The coefficients are drawn from the uniform distribution U.

$$x_t \sim \mathcal{N}(\mathbf{0}, \Sigma) \ \forall t \in \{0, 1, \ldots, T\} \tag{4.1}$$

$$\theta_i \sim U(0.1, 1) \ \forall i \in N \tag{4.2}$$

$$y_t = \theta^\top x_t \tag{4.3}$$

The covariance matrix is given using a parameter ρ to control the level of correlation between features, $\sigma_{i,j} = \rho^{min(|i-j|,|N|-|i-j|)}$. The correlation is given based on the distance $mod\,|N|$, since this makes every feature equally correlated with other features, and thus all features are symmetric. For example, the covariance matrix for four features is given by.

$$\Sigma = \begin{bmatrix} 1 & \rho & \rho^2 & \rho \\ \rho & 1 & \rho & \rho^2 \\ \rho^2 & \rho & 1 & \rho \\ \rho & \rho^2 & \rho & 1 \end{bmatrix} \tag{4.4}$$

Each synthetic dataset is generated with $T = 200$ observations. All features are present for the first 50 observations whereafter half of the features are randomly missing for each time step, rounded down. Every statistic is found by running the test several times, regenerating the data every time. This gives both a more accurate result for the MSE, as well as 95 % confidence intervals.

Both OLM and OLM-S are given fixed step sizes. For OLM the step size is set to $\tau = 0.01$, and for OLM-S the step sizes are set to $\tau_1 = \tau_2 = 0.01$, where τ_1 is the step size for updating the main weights of the classifier, while τ_2 is the step size for updating the regressions that reconstruct missing features.

4.2.1 Time horizons

First, we test the impact of time horizons on the three regression methods. The pseudocode is seen in Algorithm 6. Note that OLM-S can use time steps where features are available but targets are not yet available to train its reconstruction, and then revisit them when their targets arrive to train its predictor. Results can be seen in Figure 4.1. We see from these results that OLM-S is better than OLM when predicting on longer time horizons, likely since it has a chance to improve its reconstruction of missing features even when it cannot improve its predictor. When forecasting horizons are very short, the missing features likely act as a sort of regularization on the weights of OLM, ensuring that its predictions do not seriously deteriorate due to missing features. However, when forecasting horizons are longer, a long time passes between a feature going missing and the model training with this feature missing, which lessens this regularizing effect. OLM-S, on the other hand, performs well even without this regularizing effect, since reconstruction makes it less vulnerable to missing features. OCDS underperforms both other models, but at least seems to not deteriorate as much from longer forecasting horizons, possibly also because of its reconstruction step.

4.2.2 Correlation

I then tested the impact of different levels of correlation between features by varying ρ. I tested at a forecasting horizon of $\Delta T = 25$, since day-ahead energy markets generally require forecasting 12-36 hours ahead of time, with one prediction per hour. I followed the testing procedure in Section 6.

The results for 6 and 20 features, can be seen in Figure 4.2 and Figure 4.3, respectively. Both show a very similar result. The OLM method is slightly superior to the reconstructive OLM-S method at low correlation, then they approach and cross at about $\rho = 0.3$, and OLM-S is then superior at high correlation. OCDS never performs well compared to the other two methods. An intuitive understanding of why OLM might be better than OLM-S at low correlation is that while reconstruction should be able to improve performance even at low correlation, introducing the reconstruction step also introduces errors due to

Algorithm 6 Pseudocode for testing

```
Generate X, Y synthetically based on number of features and ρ
Initialize model m
Choose time offset ΔT
for t ∈ {1, ..., T} do
    if t + ΔT ≤ T then
        Create prediction, ŷ_t = m.predict(x_{t+ΔT})
    end if
    if using OLM-S then
        Train reconstruction on time step t + ΔT, m.train_reconstruction(x_{t+ΔT})
        Train predictor on time step t, m.train_predictor(x_t)
    else
        Train model on time step t, m.train(x_t, y_t)
    end if
end for
Evaluate model based on predictions ŷ_t for t ∈ {ΔT, ΔT + 1, ..., T}
```

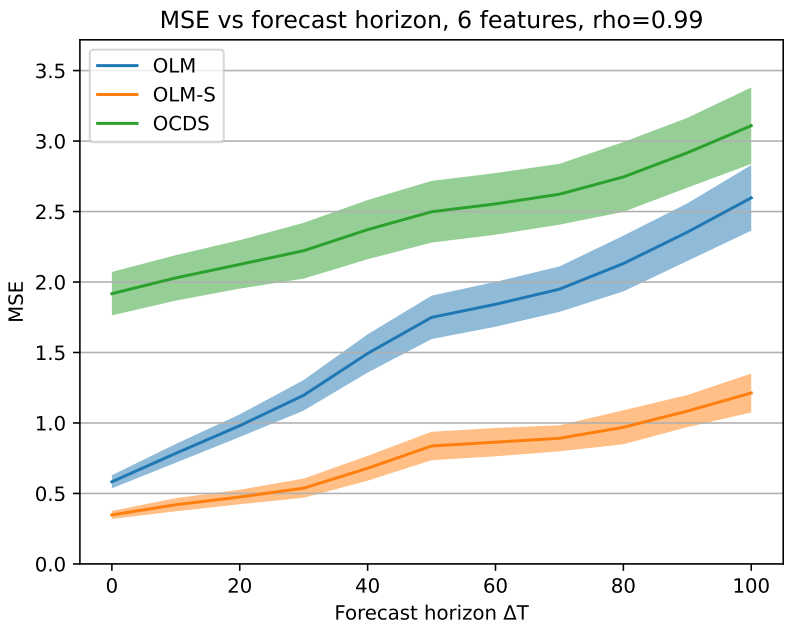


Figure 4.1: Performance at different time-ahead prediction offsets, tested according to Algorithm 6, $T = 200$, $\rho = 0.99$, 6 features, varying ΔT. 100 tests are run for every ΔT tested. Note that $\Delta T = 0$ means 1-step-ahead forecasting. We see that OLM deteriorates much faster than OLM-S as time scales get larger. OCDS does not perform well, but does approaches OLM at very long horizons.

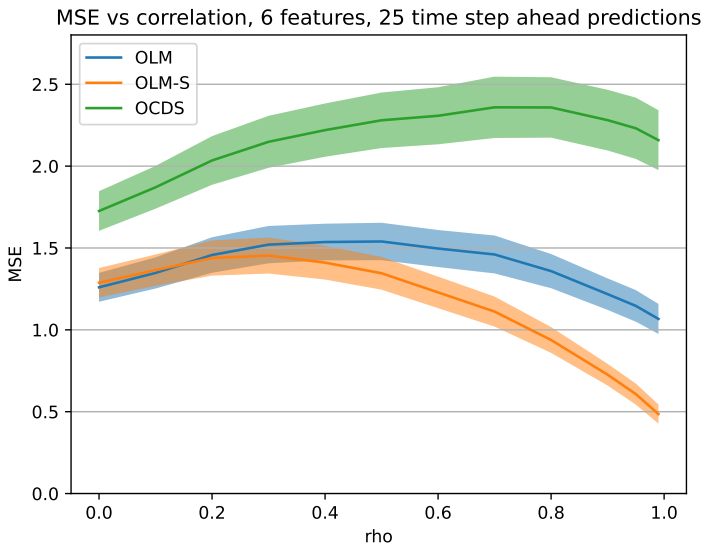


Figure 4.2: Average mean square error with 95% confidence intervals for models OLM, OLM-S, and OCDS with varying correlation, tested according to Section 6, $T = 200$, $\Delta T = 25$, 6 features, up to $\rho = 0.99$. A hundred tests are run for every tested ρ. The MSE is the expected loss of a new observation.

spurious correlations in the training data, and these spurious correlations might introduce more errors than the reconstruction improves prediction at low correlation.

We see that OLM-S improves performance on synthetic data with high correlation significantly. For example, at $\rho = 0.99$ and 6 features, the average MSE is 1.067 using OLM and 0.485 using OLM-S, or 55% lower. In comparison, at $\rho = 0$ and 6 features, the average MSE is 1.256 for OLM and 1.288 for OLM-S, which is only 2.5% lower for OLM than for OLM-S, and not a statistically significant difference. This suggests that OLM-S performs better than OLM as long as features are at least slightly correlated, and performs nearly identically at low to no correlation.

4.2.3 South Carolina Dataset

Now we test the models on a real world dataset. We use the South Carolina dataset, which is a dataset of 9 geographically collocated wind producers in South carolina. It contains the hourly production data for the 9 wind farms for the years 2007 through 2013, given for each wind farm as its current production fraction out of its maximum production capacity. The dataset is from Draxl et al. [2015]. The relative locations of the wind farms can be seen in Figure 4.4b. This is the exact type of problem we seek to construct solutions for in this thesis. It is an example of a set of wind farms that have correlated data and can improve their forecasting ability by using each other's data.

We no longer test OCDS, since it has shown to be considerably worse than OLM and OLM-S in every case tested so far.

First we examine the dataset, since we want to know whether we expect that OLM-S can perform well on this dataset. Therefore, we examine the inter-feature correlation, as seen in Figure 4.4a. This correlation looks to be in the high-correlation domain where OLM-S outperformed OLM on the synthetic dataset. We therefore expect OLM-S to outperform OLM on this dataset as well when the wind electricity production actuals from the dataset are used as features.

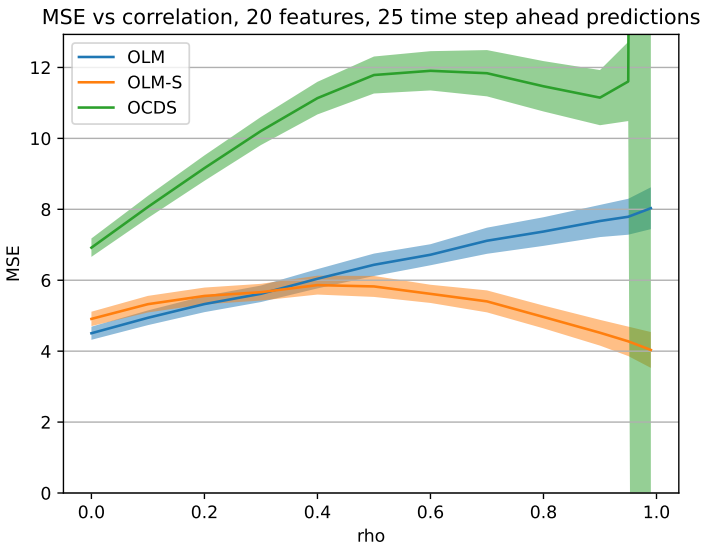
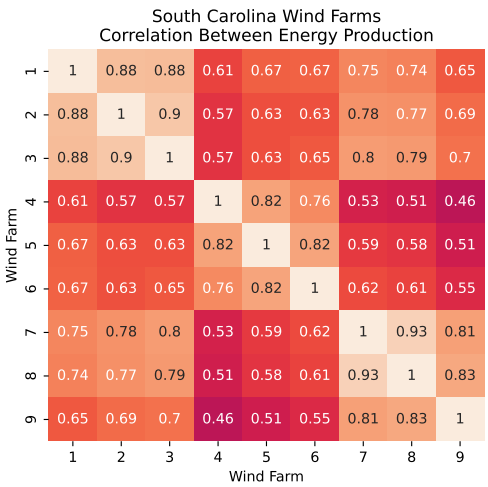
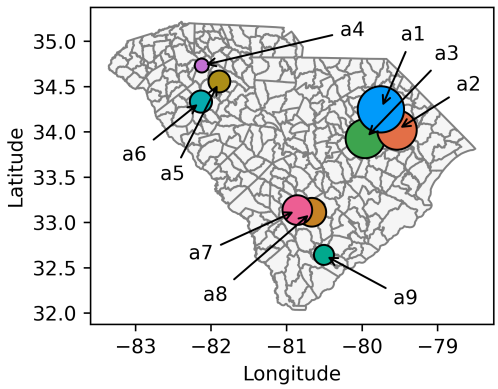


Figure 4.3: Average mean square error with 95% confidence intervals for models OLM, OLM-S, and OCDS with varying correlation, tested according to Algorithm 6, $T = 200$, $\Delta T = 25$, 20 features, up to $\rho = 0.99$. A hundred tests are run for every tested ρ. With 20 features, the errors introduced by spurious correlations are so big that OLM-S is statistically significantly worse than OLM at $\rho = 0$. OCDS experiences numerical issues at very high correlation which results in the explosion seen at around $\rho \in [0.95, 0.99]$.



(a) Correlation between % wind production in the 9 wind farms in the South Carolina dataset.



(b) Locations of the 9 wind farms represented in the South Carolina dataset.

Figure 4.4: Basic information about the South Carolina dataset. The wind farms are geographically organized into three clusters, which also shows in the inter-feature correlations. We then expect OLM-S to outperform OLM on this dataset.

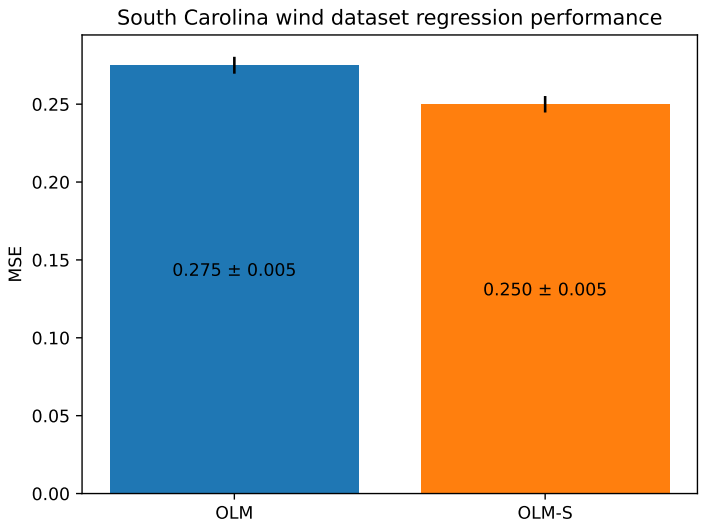


Figure 4.5: Average MSE over 100 start times on the South Carolina dataset using OLM and OLM-S. 900 runs of the market, one for each start time and wind farm. 1 step ahead forecasting. OLM-S results in a statistically significant decrease in loss compared to OLM. Note that due to erratic results on early time steps, the MSE is only calculated from observation 60 onwards, 1/4 through the dataset. OLM and OLM-S are equivalent on these first 60 time steps since no features are missing.

We then test the two models, comparing their regression performance on the dataset. We forecast production one hour ahead as introduced in Section 2.2.2. This forecast is useful for wind producers when determining their position in the intraday market, where they can modify their position from the position they took in the day-ahead market using the most up-to-date information available. This test then lets us understand the usefulness of the OLM and OLM-S regresion methods in a realisitic forecasting situation.

We compare mean squared error, since this is the loss function we minimize. For every test, we pick 240 observations from the dataset, equivalent to ten days of observations. We take one of the 9 wind farms and use its production as the regression target and use the production data of all nine wind farms as features with a single hour of lag. We normalize all features. From halfway through the test, we randomly choose half of the features every time step to go missing. For every start time chosen, we run the regression through for every target feature.

The results can be seen in Figure 4.5. We see that OLM-S outperforms OLM in this realistic forecasting situation, indicating that there is improvement to be found by using OLM-S for real world regression markets.

4.2.4 Discussion of OCDS

The results for the OCDS method were quite a lot worse than the baseline OLM model, and not as good as what was reported in the original paper. Therefore, we investigate some of the choices made by the OCDS algorithm, to understand why it underperforms.

a) Loss function

OCDS uses Mean Squared Error loss for classification. I've changed this in my implementation for Binary Cross-Entropy loss, since MSE loss for classification means that a

classifier could make a correct prediction, but be penalized more for that than it would be for making an incorrect prediction, e.g. if it predicted $\hat{y}_t = 3$ and $y_t = 1$, even though it predicted the correct label, sign$(\hat{y}_t) = 1$.

b) Step size

OCDS is meant to work for arbitrarily changing feature spaces. This means that it should not matter when a feature first enters the model. Yet, OCDS uses a step size that decreases over time. Imagine that only one feature is present until time step 100, where a new features enter the model. This new feature has a much lower step size than the original feature did when it appeared, meaning that OCDS will treat features differently based on when they enter the model. This observation also motivates the fixed step sizes presented for OLM and OLM-S in this thesis, though further work investigating changing step sizes in variable feature spaces and non-stationary distributions could be valuable.

c) Ensembling Method

The present and reconstructed feature predictions are ensembled together by a parameter p, where a prediction based on the present features and reconstructed features are produced independently and then weighed together, $\hat{y} = p\hat{y}_{present} + (1-p)\hat{y}_{reconstructed}$. The parameter p is not at all dependent on the number of features present. This means that if all features are present, the model prediction becomes $p\hat{y}_{present} + (1-p) \cdot 0$, i.e. it receives a vote of 0 with weight $1-p$ for no other reason than no features being missing.

d) G matrix dynamics

The G matrix is trained on the reconstruction error $\|\Pi_{\mathbb{R}^{d_t}}(\frac{1}{|d_t|}Gr_t x_t) - x_t\|_2^2$. However, this means that if the same features are present for a long time, the matrix Gr_t will converge to $|d_t|Ir_t$, where Ir_t is the columns d_t of the identity matrix I so that $Ir_t x_t$ is equal to $x_{i,t}$ on all indices $i \in d_t$ and 0 on all indices $i \in u_t \setminus d_t$, since the error will then be

$$\|\Pi_{\mathbb{R}^{d_t}}(\frac{1}{|d_t|}|d_t|Ir_t\, x_t) - x_t\|_2^2 = \|\Pi_{\mathbb{R}^{d_t}}(Ir_t\, x_t) - x_t\|_2^2 = \|x_t - x_t\|_2^2 = 0 \tag{4.5}$$

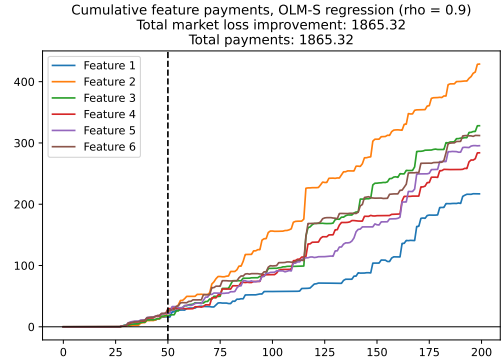
This means that the G matrix does not necessarily converge towards a useful reconstruction function; in fact, if all features are present up until some time step t_1, the matrix will at time step $t_1 - 1$ be converging towards the identity matrix I times the number of features present $|u_{t-1}|$. This also motivated the reconstruction method used in OLM-S, which avoids this dynamic by not training the subregression reconstructing a certain feature on the feature itself.

There are more choices worth analyzing within OCDS, but I will end my analysis here for brevity.

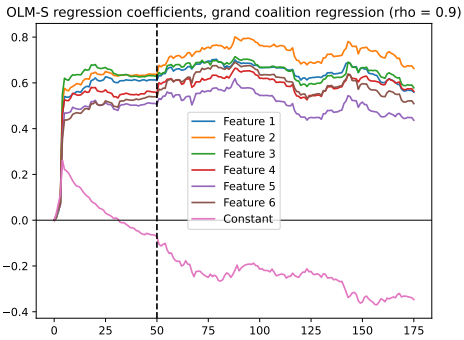
4.3 Regression Market

We conclude that OLM and OLM-S are the best choices available for constructing an online regression market with varying feature space. We can construct a market with either of the two models. We expect OLM to perform better when features are not very correlated, and OLM-S to perform better when features are more highly correlated.

In this section we examine results from running a regression market using the OLM and OLM-S models as market models. We compare using reconstruction as seen in OLM-S



(a) Cumulative payments over time for the sold features in the synthetic dataset.



(b) Coefficients of the features over time. Feature 7 is the constant feature owned by the central agent. Note that the coefficients in the model are not in fact tracking the true coefficients underneath. This is because of inter-feature correlation and missing features.

Figure 4.6: An example run of a regression market on a synthetic dataset generated as described earlier, following the method in Algorithm 6, $T = 200$, $\Delta T = 25$, 6 features, using the OLM-S regressor. Shows both cumulative payments over time as well as the regression coefficients for the six sold features and the constant feature.

with the baseline OLM. We also investigate the structure of the revenue allocation including instances of negative payment, missing features, how revenue is increased from feature reconstruction, and the effects of mean imputation on revenue allocation.

The always-present constant feature is given to the central agent as its only feature.

4.3.1 Running the Market

In this section, we test the models OLM and OLM-S as market models in situations with missing data. We use a setup as seen in the regression section, Algorithm 6. We set the buyer's willingness to pay to 1. We generate the synthetic datasets and run the market several times to generate 95% confidence intervals for statistics.

An example run of the market is shown in Figure 4.6. We see the cumulative payouts for each feature, as well as how the coefficients of the model change over time. The missing features are reconstructed each time step. Note that as coefficient estimates improve over time, payments also increase, as the value of features increases.

To investigate the out-of-sample outcome of using OLM-S versus OLM in the market, we run the market several times. The average feature payments using market models OLM and OLM-S in a high correlation context are shown in Figure 4.7. All features are paid better using reconstruction than without reconstruction. Note that if all market participants mean impute, the OLM-S method reduces to OLM. We then expect that market participants are incentivized to truthfully report their features as missing, since OLM-S pays better. We investigate this further in Section 4.3.5. However, when correlations are low, we know that OLM sometimes outperforms OLM-S, so participants might be incentivized to mean impute when their feature does not correlate much with other features. Then, truthfulness under imputation might not always hold at low correlation using OLM-S as the market model in out-of-sample situations.

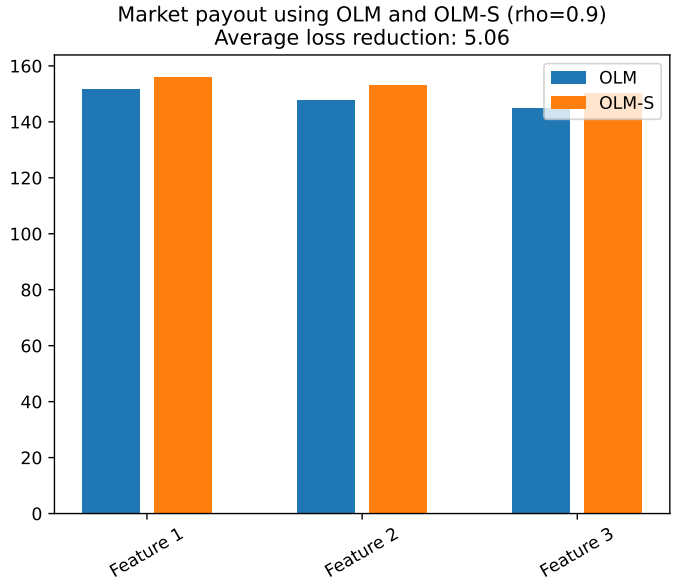


Figure 4.7: Average payment for each feature over 100 runs of the market, generating predictions using Algorithm 6, $T = 200$, $\Delta T = 25$, 3 features. The difference in the mean payments for the features are all statistically significant in a paired t-test with $p < 10^{-10}$. While the differences here are not large percentage-wise, they could still be large in absolute terms if the willingness to pay was very large. Even small differences can also change profit margins considerably.

4.3.2 Negative payments

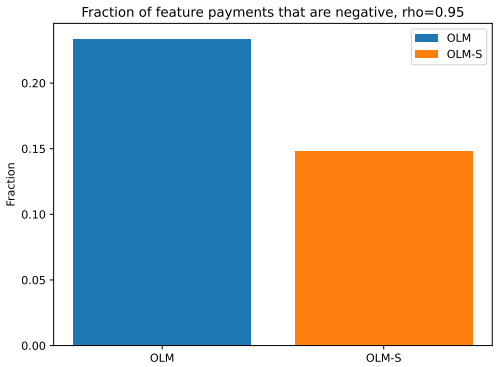
Feature owners sometimes receive negative payments, when they have negative Shapley values. The fraction of negative payments and the sum negative payments allocated by OLM and OLM-S are shown in Figure 4.8. We see that the risk of having to pay back to the market is much smaller when using OLM-S as the regression mechanism. For a risk-averse agent, this is a bonus of using OLM-S as the market mechanism, and brings us closer to an ex-post individually rational market.

4.3.3 Payment for missing features

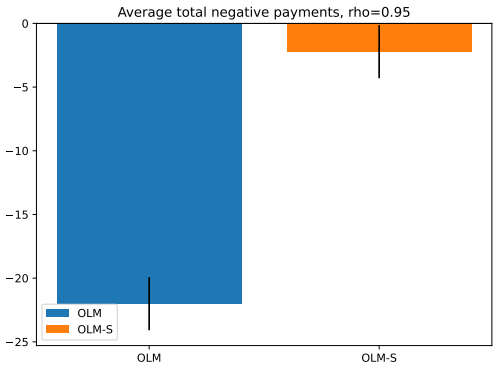
Features still get Shapley values assigned to them when they are missing, and thus still receive payments. We examine the payment received by missing features in Figure 4.9. We see that the scale of payments for missing features are very small. We also see that feature owners are much more likely to receive a negative payment when their feature is missing, and for OLM they actually expect negative payments both on average and on most time steps.

We see that using OLM-S makes the payments to missing features less negative, and actually makes them positive on average. This is favorable since it means less risk for market participants. While payments for missing features are positive on average using OLM-S, they are still very small compared to payments for present features. While it might seem to be unfair to pay features that are not present, I argue that this is fair since the increase does still originate from an increase in predictive performance due to that feature being present before. If the increase is only caused by a regularizing effect, though, then a market with appropriate regularization wouldn't pay this positive payout to missing features.

The expected in-sample payment for a missing feature at the time step when it is missing

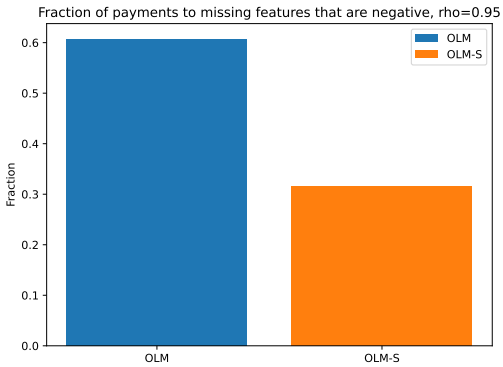


(a) Fraction of all payments to feature sellers that are negative.

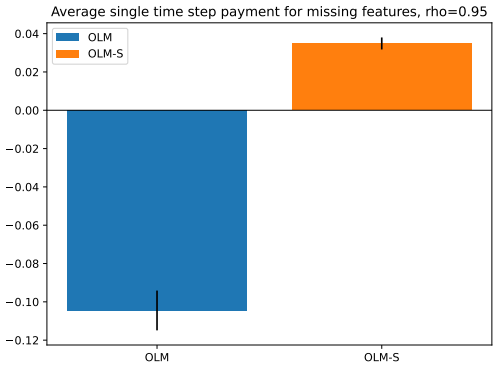


(b) Total negative payments given out in a single run of the market.

Figure 4.8: Fraction of all payments that are negative and total negative payment per run, over 100 runs of the market, generating a synthetic dataset and using Algorithm 6, $T = 200$, $\Delta T = 25$, 3 features. For comparison, the total payment given out in a single run of the market is 455 on average for OLM, and 471 for OLM-S.



(a) Fraction of single time-step payments to missing features that are negative.



(b) The average single time-step payment to missing features across all time steps and runs. Error bars indicate the standard deviation.

Figure 4.9: Payments for missing features for a hundred runs of the market using a synthetic dataset. Following Algorithm 6, $T = 200$, $\Delta T = 25$, 3 features. For comparison, every time step that a feature is present in the market, it is paid on average 0.98 by OLM and 0.96 by OLM-S, meaning that payments for missing features are about a magnitude or two smaller.

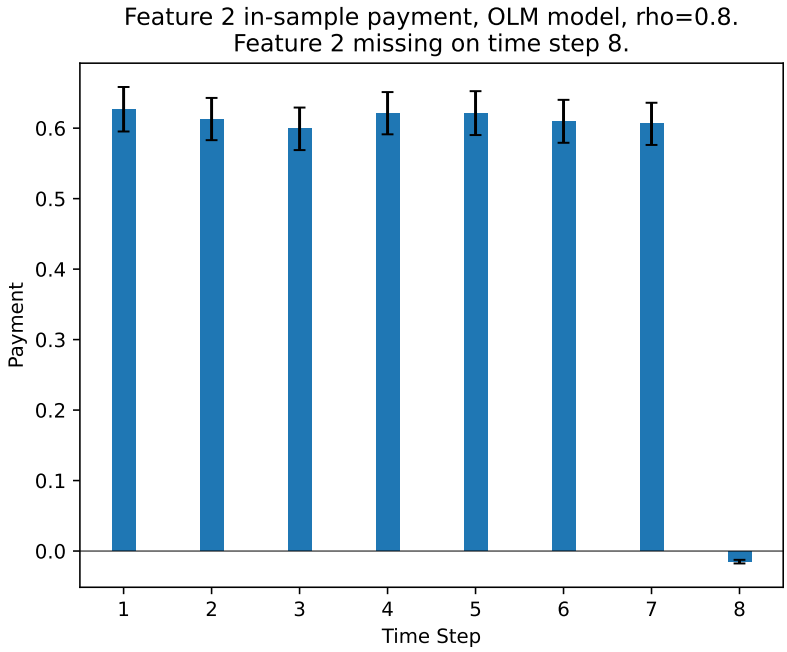


Figure 4.10: In-sample payment to feature 2 over 10000 runs using an in-sample version of the OLM, i.e. ordinary linear regression with mean imputation. Two features, eight time steps, no constant feature. Features are drawn from a centered normal distribution, $\rho = 0.8$. True coefficients are drawn from a uniform distribution $U(0.1, 1)$. Feature 2 is present on time steps 1-7, and missing on time step 8, which causes it to have a negative Shapley value at time step 8 in-sample in-expectation. Feature 1 is always present

is entirely dependent on the distribution of the data and the distribution of the missing values, so we can only evaluate this payment in specific situations. This allows us to understand what payments to expect for feature owners when their features are missing. We investigate the in-sample average payment for missing data at fixed time steps using the in-sample regression version of the OLM, i.e. an ordinary linear regression where missing features are mean imputed as 0.

We use just two features. We draw true coefficients from the uniform distribution $\theta_i \overset{iid}{\sim} U(0.1, 1) \forall i \in \{1, 2\}$. The features are drawn from a centered multivariate Gaussian distribution $x_t \overset{iid}{\sim} \mathcal{N}(0, \Sigma)$. The true targets are given by $y_t = [\theta_1, \theta_2]x_t$, and the regression is performed on x_t but with feature 2 missing for some time steps (which is equivalent to setting it to zero).

The results are shown in Figure 4.10. We see that feature 2 has a positive Shapley value when present, but then receives a small negative Shapley value when it is missing. We can learn two things from this:

1. Feature owners expect significantly higher payments when their features are present than when they are not, and thus are incentivized to keep their feature present

2. In some situations using the OLM model, feature owners get negative payments even in expectation when their features are missing.

4.3.4 Revenue Impact from Reconstruction

Feature revenue is increased in OLM-S from reconstruction. Using a Shapley value approach we can decompose the gained revenue for each feature by each other feature being reconstructed. We construct a cooperative game where the payoff is the revenue for feature i, while the coalitions are the set of features that are reconstructed. The features that are not in the coalition are mean imputed. This cooperative game then shows how much feature i earns when the features ω are being reconstructed, so finding the Shapley value for feature j shows the impact on the revenue of feature i from feature j being reconstructed. This approach is described in Algorithm 7. Remember that N is the set of all players in the game, i.e. all features in the market.

Algorithm 7 Pseudocode for decomposing revenue gain by feature.

Generate X, Y synthetically based on number of features and ρ
for $\omega \subseteq N$ **do**
  Initialize OLM-S model m_ω that only reconstructs features ω and mean imputes features $N \setminus \omega$.
  Calculate sum payment $\pi_{i,\omega}$ for each feature i using model m_ω and dataset X, Y.
end for
for $i \in N$ **do**
  Calculate Shapley value $\psi_{i,j}$ for each feature j where the payoff for each coalition ω is $\pi_{i,\omega}$
end for
Shapley values $\psi_{i,j}$ show contribution to payment to feature i from the algorithm reconstructing feature j

The revenue breakdown from reconstructing each feature is shown in Figure 4.11. We see that a feature has its revenue considerably increased by being reconstructed, but decreased by other features being reconstructed. Since we know that payments are very small at time steps where features are missing, this indicates that reconstruction keeps feature coefficients of high quality while a feature is missing, so that it can continue performing when it reenters the market. Reconstruction is thus a service that the rest of the features perform for the missing feature. This could be a motivation for the allocation idea presented in Section 3.3.1, where the Shapley value of a missing feature is redistributed between the present features, but that actually doesn't entirely track, since the increase in feature revenue from being reconstructed is not the same as their payment when missing.

4.3.5 Truthfulness under Imputation

When a feature is missing, participants in the market have the option to mean impute their missing information based on historical data. This is uninformative, and can worsen the market performance.

Mean imputation has no effect on the OLM model, and reverts OLM-S to OLM. As we saw in Section 4.2.2, the OLM model outperforms OLM-S in some low correlation situations.

We expect that features are disincentivized from mean imputing in a high correlation setting from the revenue impact in Figure 4.11. The fact that OLM outperforms OLM-S in low correlation points to features being incentivized to mean impute at low correlation. To be certain in both low and high correlation, we investigate the difference in payment for every feature if they mean impute versus if they do not at both low and high correlation. This is shown in Figure 4.12. This shows that at low correlation, the difference in revenue is not significant, while at high correlation, every feature loses revenue by mean imputing. Then for the synthetic dataset at high correlation, truthful reporting is an equilibrium since

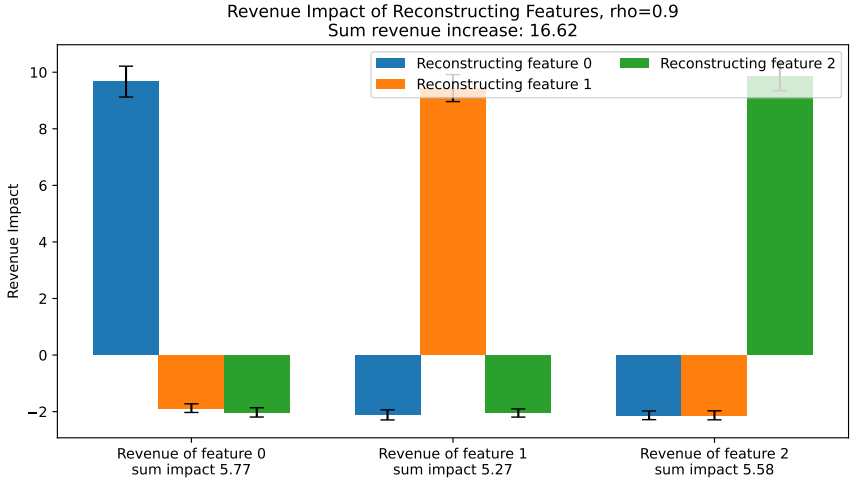


Figure 4.11: Impact to feature revenue from reconstructing other features, calculated by Algorithm 7. From 100 runs of the market, using Algorithm 6, $T = 200$, $\Delta T = 25$, 3 features.

no feature can increase their own revenue by mean imputing, and the high correlation market is truthful under imputation in practice. In the low correlation market, we cannot determine if the market is truthful under imputation based on these results.

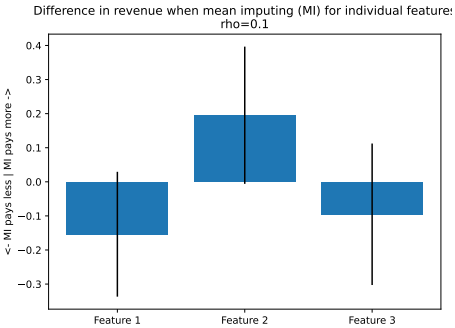
For OLM-S, we do not establish guarantees for truthfulness under imputation. Feature owners will have to determine whether to mean impute their feature or not when interacting in the market, which depends on the other features available in the market and the distribution of their missing features. We also see in Figure 4.11 that features do have their revenue increased from being reconstructed in high correlation settings, but actually have their revenue decreased from other features being reconstructed. Then, every feature owner would prefer to be the only feature that is reconstructed. They do not have control over this, and so this does not cause unintended incentives in the market. One feature owner imputing their feature will seemingly not damage the gains of other feature owners, though, meaning that a feature owner cannot use this maliciously, either.

Theoretical bounds and guarantees for truthfulness under imputation are not constructed in this thesis. The payment method where Shapley values are calculated only based on present features is intended to solve truthfulness under imputation since missing features often have negative or near-zero Shapley values in expectation. However, as mentioned before, this allows features to deteriorate the market without paying for it by going missing, thus breaking the Shapley fairness criteria.

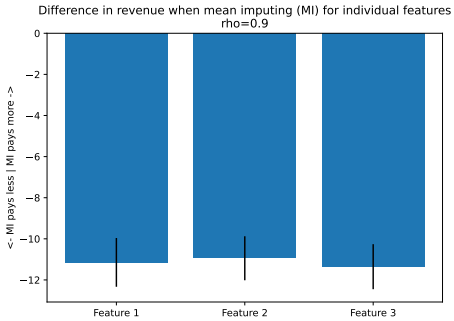
4.3.6 South Carolina Dataset

In this section, we test the market when the forecasting problem is as described in Section 4.2.3 for the South Carolina wind farm dataset. When running the market, we will choose a start time and run the market nine times for that start time, once predicting the output of each wind farm.

The results are shown in Figure 4.13. We see how almost all features earn more from using the reconstructive market model, except features 4 and 9. To investigate whether all features will truthfully report their features as missing, with special interest in features 4 and 9, we will examine their difference in revenue when mean imputing. This can be seen in Figure 4.14. We see that even though features 4 and 9 do not have their revenue



(a) Difference in revenue when mean imputing at low correlation, showing no significant difference for any feature.



(b) Difference in revenue when mean imputing at high correlation, showing that all features have their revenue significantly reduced by mean imputing when all other features are reconstructed.

Figure 4.12: Difference in sum market revenue for each feature for mean imputing when all other features do not, using the OLM-S market model. 100 runs of the market, $T = 200$, $\Delta T = 25$, 3 features, synthetic dataset. Using Algorithm 6.

increased from using OLM-S as the market model, they and all other feature owners still lose revenue by mean imputing, so the participants are incentivized to report their missing features truthfully. This might be slightly confusing, since we know that OLM-S becomes equivalent to OLM when features mean impute; however, this is only true when all features are mean imputing. The revenue difference seen in Figure 4.14 is the revenue difference when a single feature decides to mean impute instead of truthfully reporting their features as missing.

The wind farm owners would then be better off selling data to one another through this market. With a functional solution for the buyer side, this market would successfully increase the revenue of every wind farm. The market would continue running if one wind farm was unable at one hour to provide its production information to the other farms, for whatever reason that might be, or if new wind farms were to join the market, or old farms were to leave. We can then use our market that supports missing features to loosen up the constraints on regression markets, facilitating mutually beneficial data exchange and improving social welfare.

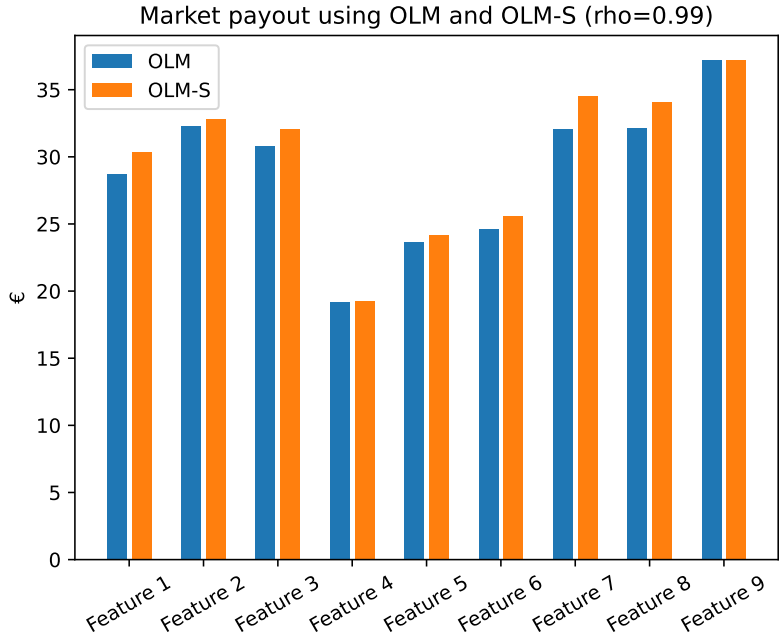


Figure 4.13: Average total revenue using the OLM and OLM-S models on the South Carolina dataset like in Section 4.2.3. 10 different starting points, 90 runs of the market, 1-step-ahead predictions. All features except feature 4 and 9 have increased revenue from using OLM-S.

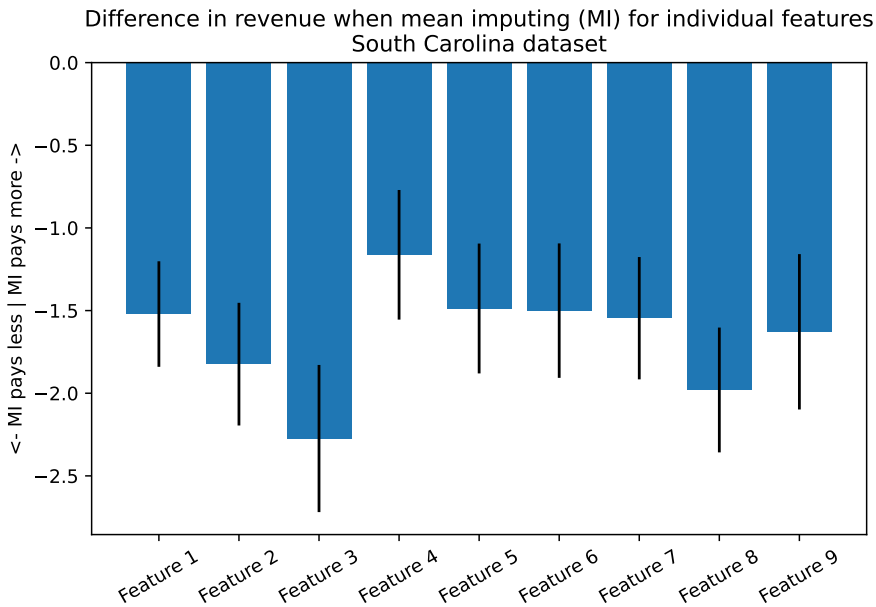


Figure 4.14: Difference in market revenue for each feature for mean imputing when all other features truthfully report their features as missing, Using the OLM-S market model. South Carolina datset, 10 start times, 90 runs of the market (1 for each start time and column), same setup as in Figure 4.13. We see that all features have their revenue reduced when mean imputing, so truthful reporting of missing features is an equilibrium strategy, and we have truthfulness under imputation.

5 Conclusions

In this work we explored analytics markets as a method for incentivizing data sharing for forecasting problems, focusing on renewable energy forecasting in particular. We relaxed the assumption in existing literature that the features in the market are fixed over time, as this represents a key barrier to the practical implementation of analytics markets.

We have investigated current research into machine learning models that support varying feature spaces and found that existing methods in utilitarian online learning literature did not live up to their reported performance. This motivated us to develop a new regression model as a contribution to this field. The model supports varying feature spaces and improves regression accuracy in situations with high correlation, and is known as the Online Linear Model with Subregressions (OLM-S). We have shown that both OLM-S and a simpler online linear model (OLM) can be used for regression markets with varying feature spaces, using the Shapley value allocation policy.

We have investigated the dynamics of the Shapley value allocation policy when features are missing, and found that the OLM model often assigns negative values to missing features. We have found that OLM-S does not do this as much, thus bringing the market closer to being ex-post individually rational, a desirable property of analytics markets that incentivizes sellers to participate in the market. More work can be done to make the market closer to ex-post individually rational, as will be discussed below.

We have also found that there is a new strategy space when features are missing, namely whether feature owners impute their missing value or not. This results in a new property, truthfulness under imputation. A market setup that is truthful under imputation will have all feature owners truthfully report their features as missing, instead of reporting an imputation. In this thesis, we only consider mean imputation as an imputation method, and leave the consequences of other imputation types as further work. Truthfulness under imputation holds weakly with the OLM, but it is not established for the OLM-S model.

We have used both the OLM and OLM-S models on synthetic and realistic regression tasks. We find that both can result in working markets and that our proposed model, OLM-S, can increase revenue over OLM in realistic market situations where we have truthfulness under imputation.

Our regression model proposed here has application beyond analytics markets in general online machine learning tasks. Typically in real-world regression tasks, only few features are randomly missing at one point in time compared to the total amount of features. On the other hand, it is common for the number of features available to change over time, i.e. new features appearing and old features disappearing permanently. Methods dealing with varying feature spaces are useful in this case, and the results in this thesis could be applied, though a more focused handling of appearing and disappearing features would then be relevant.

The online setting likely complicates reconstruction of missing features. High-quality in-sample imputation methods like Fully Conditional Specification along with retraining the model for only the features present at prediction time could likely improve the quality of predictions in varying feature spaces. Additionally, retraining at prediction time ensures a Shapley value of zero for missing features, and thus removes the question of whether negative and positive payments to missing features is really fair. These are avenues for

further research as described below, and together could form the basis of a well-working regression market with varying feature space.

5.1 Future Work

In this section, we will identify some avenues for further research.

5.1.1 Ex-post Individual Rationality

The market presented in this thesis is not ex-post individually rational. This means that while all participants expect positive returns, they cannot be sure that they will receive positive returns at every run of the market. This dissuades risk-averse participants from entering the market. However, even though every feature seller is almost guaranteed to eventually receive negative payments if their feature occasionally goes missing, any informative feature is also almost guaranteed to receive positive payments from the market in total over the course of the market. A solution could then be to use expected Shapley values using a running average or similar, like what is used in Pinson et al. [2022], and work on out-of-sample guarantees could possibly establish full ex-post individual rationality, or bounds on the proportion of payments that might be negative.

5.1.2 Other Types of Feature Spaces

All results in this thesis are about data streams where features are randomly missing. A natural question would be how these results hold up in data streams where new features appear or old features disappear permanently. In real-world forecasting tasks within renewable electricity production, new features appearing and old features disappearing is common.

The problem of new features appearing is a simple one, since the new feature simply does not receive any Shapley value until it appears. Then, when it appears, it is in the market on the same level as every other feature. The only issue to deal with with these new features is that of warm up time. We expect models to be pretty poor at early time steps, since they don't have a lot of information about the coefficients of the model. This can lead to wildly erratic payments, and is usually dealt with by setting a warm up time where the model is training but no payments are issued. This can be implemented for trapezoidal streams, where it would be possible to keep two models running when new feature(s) enter, one with the old set of features and one where the new feature(s) are included, and only switch to the new model once the new feature(s) have passed their warm-up time. The payments can also be made less erratic by introducing regularization.

Disappearing features are more interesting, since after disappearing, they will have a "tail" of payments that might all be negative. It would then be relevant to investigate the effect of market and model choices on this tail. One could e.g. imagine that a model with decreasing step size over the existence of a feature might be useful, but detrimental for a feature that has disappeared, since its former existence would worsen the market for a long time. It might also be necessary to set a sort of timeout on features, where features that have been missing for more than a certain number of time steps are entirely removed from the model, to limit both their losses and their negative effect on the market model.

5.1.3 Batch Learning and Variable Features

As mentioned earlier, using an online learning model for multi-horizon forecasting is similar to using a batch learning model that is occasionally retrained. Using batch learning would allow work on batch learning markets to more easily translate to varying feature spaces. It would also allow more sophisticated imputation methods like Fully Conditional Specification. A natural avenue to pursue for further study would then be to compare

the performance of a batch learning model using e.g. Fully Conditional Specification to handle missing features with the OLM-S model presented in this paper.

5.1.4 Retraining vs imputation

The paper Tawn et al. [2020] finds that in practice, retraining a regressor instead of imputing missing features at prediction time actually provides superior performance for the market model. The only issue is that this can be computationally very expensive since the model can theoretically need to be retrained for every set of features if the set of missing features changes often.

However, this method allows use of the payment allocation policy where only present features receive payment in the market, since missing features will always receive a Shapley value of zero as they do not influence the model. This removes the risk of providing missing features to the market and would be a very good avenue for research to improve the behavior of variable feature space markets. Truthfulness under imputation would be important to investigate for a market of this type since feature owners know that they receive no payment when they are not present.

5.1.5 Theoretical Work for Truthfulness Under Imputation

Truthfulness under imputation is relevant for any regression or classification market in varying feature space that uses reconstruction to handle missing features. This is true whether the model is online or meant for batch learning, and whichever reconstruction model is used. Therefore, it could be useful to establish more theoretical bounds and guarantees on truthfulness under imputation, and to build up a framework for evaluating truthfulness under imputation for other markets in varying feature space when and if these are constructed.

Seller-side imputation in this thesis is limited to mean imputation, but sellers have many other imputation methods at their disposal. They could use regression imputation similar to what is used in the market, or autoregressive imputation, or use some other knowledge they have available to reconstruct their features. At this point, we might actually want sellers to provide their own imputations; even though they likely wouldn't be as good as the true value of their feature, the sellers might still improve regression performance compared to the imputation used by the market. It might then be relevant to split truthfulness under imputation into properties based on the exact type of imputation that the market is truthful under, e.g. truthfulness under mean imputation.

When feature sellers impute their feature, they are in effect selling predictions of their missing feature. This interaction could be investigated from the view of prediction markets, where the incentives of selling predictions have already been worked through.

5.1.6 Probabilistic and Bayesian Models

The optimal bid in electricity markets is often a function of the probability distribution of the forecasted power output. (Hong et al. [2020]) This is a motivation for probabilistic models which model the entire conditional distribution instead of a specific charactersistic of it, e.g. the expected value in our case. The Online Linear Model presented in this thesis can naturally be extended to a probabilistic model by using the Maximum Likelihood Estimation specification of the model and estimating the noise variance along with the model parameters.

The probabilistic version of OLM-S could be improved beyond simply estimating the noise variance when extending to a probabilistic model, since the subregressions can also output a probabilistic estimate of the missing features. To effectively model the uncertainty

due to imputation, one would need to propagate these noise estimates to the main regression problem.

Uncertainty propagation also gives us another reason why truthfulness under imputation is useful: When features are truthfully reported as missing, the deterioration in predictive quality can accurately be reported through the market, e.g. by increased uncertainty in a probabilistic forecast. When feature sellers impute their feature themselves, this uncertainty cannot be propagated, and the probabilistic forecast might be overly confident compared to what the data suggests. If one does not seek truthfulness under imputation as mentioned in Section 5.1.5, another option would be to let feature sellers report their features as a probability distribution, e.g. as an expected value and standard deviation. Thus, feature sellers could accurately report a self-produced imputation including uncertainty to the market, and be rewarded for the improvement in the prediction quality.

Another natural development would be to use Bayesian linear regression where coefficients also have an uncertainty attached to them. These models can naturally be extended to the online setting, since the Bayesian update step used for fitting these models can be used on an entire dataset or a single observation at a time.

Probability distribution estimation requires some more work on the market side regarding market properties and revenue allocation, but this work has already been done in Falconer et al. [2024b].

Bibliography

Tao Hong, Pierre Pinson, Yi Wang, Rafał Weron, Dazhi Yang, and Hamidreza Zareipour. Energy forecasting: A review and outlook. *IEEE Open Access Journal of Power and Energy*, 7:376–388, 2020. doi: 10.1109/OAJPE.2020.3029979.

José R. Andrade and Ricardo J. Bessa. Improving renewable energy forecasting with a grid of numerical weather predictions. *IEEE Transactions on Sustainable Energy*, 8(4): 1571–1580, 2017. doi: 10.1109/TSTE.2017.2694340.

Laura Cavalcante, Ricardo J. Bessa, Marisa Reis, and Jethro Browell. Lasso vector autoregression structures for very short-term wind power forecasting. *Wind Energy*, 20 (4):657–675, 2017. ISSN 10991824, 10954244. doi: 10.1002/we.2029.

Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *Ieee Signal Processing Magazine*, 37(3):50–60, 2020. ISSN 15580792, 10535888. doi: 10.1109/MSP.2020.2975749.

Esther Gal-Or. Information sharing in oligopoly. *Econometrica*, 53(2):329–343, 1985. ISSN 00129682, 14680262. URL http://www.jstor.org/stable/1911239.

Magdalena Balazinska, Bill Howe, and Dan Suciu. Data markets in the cloud: An opportunity for the database community. *Proceedings of the Vldb Endowment*, 4(12): 1482–1485, 2011. ISSN 21508097.

Zicun Cong, Xuan Luo, Jian Pei, Feida Zhu, and Yong Zhang. Data pricing in machine learning pipelines. *Knowledge and Information Systems*, 64(6):1417–1455, 2022. ISSN 02193116, 02191377. doi: 10.1007/s10115-022-01679-4.

Yiling Chen and David M. Pennock. Designing markets for prediction. *Ai Magazine*, 31 (4):42–52, 2010. ISSN 23719621, 07384602. doi: 10.1609/aimag.v31i4.2313.

René Aïd. Electricity markets. *Electricity Derivatives*, pages 5–25, 2015. doi: 10.1007/978-3-319-08395-7_2.

Julija Tastu. Short-term wind power forecasting: probabilistic and space-time aspects. 2013.

Michael Raith. A general model of information sharing in oligopoly. *Journal of Economic Theory*, 71(1):260–288, 1996. ISSN 00220531, 10957235. doi: 10.1006/jeth.1996.0117.

Nikita Tsoy and Nikola Konstantinov. Strategic data sharing between competitors. *Advances in Neural Information Processing Systems*, 36, 2023. ISSN 10495258.

Dirk Bergemann and Alessandro Bonatti. Markets for information: An introduction. *Annual Review of Economics*, 11(1):85–107, 2019. ISSN 19411391, 19411383. doi: 10.1146/annurev-economics-080315-015439.

Mohammad Rasouli and Michael I. Jordan. Data sharing markets, 2021. URL https://arxiv.org/abs/2107.08630.

Carla Goncalves, Pierre Pinson, and Ricardo Jorge Bessa. Towards data markets in renewable energy forecasting. *Ieee Transactions on Sustainable Energy*, 12(1):533–542, 2021. ISSN 19493029, 19493037. doi: 10.1109/TSTE.2020.3009615.

Pierre Pinson, Liyang Han, and Jalal Kazempour. Regression markets and application to energy forecasting. *Top*, 30(3):533–573, 2022. ISSN 18638279, 11345764. doi: 10.1007/s11750-022-00631-7.

Liyang Han, Pierre Pinson, and Jalal Kazempour. Trading data for wind power forecasting: A regression market with lasso regularization. *Electric Power Systems Research*, 212: 108442, 2022. ISSN 18732046, 03787796. doi: 10.1016/j.epsr.2022.108442.

Xuanyu Cao, Yan Chen, and K. J. Ray Liu. Data trading with multiple owners, collectors, and users: An iterative auction mechanism. *Ieee Transactions on Signal and Information Processing Over Networks*, 3(2):268–281, 2017. ISSN 23737778, 2373776x. doi: 10.1109/TSIPN.2017.2668144.

Daron Acemoglu, Ali Makhdoumi, Azarakhsh Malekian, and Asu Ozdaglar. Too much data: Prices and inefficiencies in data markets. *American Economic Journal: Microeconomics*, 14(4):218–256, 2022. ISSN 19457685, 19457669. doi: 10.1257/mic.20200200.

Anish Agarwal, Munther Dahleh, Thibaut Horel, and Maryann Rui. Towards data auctions with externalities. 2023.

Lloyd S Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker, editors, *Contributions to the Theory of Games II*, pages 307–317. Princeton University Press, Princeton, 1953.

Hugh Chen, Scott Lundberg, and Su-In Lee. *Explaining Models by Propagating Shapley Values of Local Components*, pages 261–270. Springer International Publishing, Cham, 2021. ISBN 978-3-030-53352-6. doi: 10.1007/978-3-030-53352-6_24. URL https://doi.org/10.1007/978-3-030-53352-6_24.

Anish Agarwal, Munther Dahleh, and Tuhin Sarkar. A marketplace for data: An algorithmic solution. *Acm Ec 2019 - Proceedings of the 2019 Acm Conference on Economics and Computation*, pages 701–726, 2019. doi: 10.1145/3328526.3329589.

Thomas Falconer, Jalal Kazempour, and Pierre Pinson. Towards replication-robust analytics markets, 2024a. URL https://arxiv.org/abs/2310.06000.

Honglin Wen, Pierre Pinson, Jie Gu, and Zhijian Jin. Wind energy forecasting with missing values within a fully conditional specification framework. *International Journal of Forecasting*, 40(1):77–95, 2024. ISSN 18728200, 01692070. doi: 10.1016/j.ijforecast.2022.12.006.

S. Van Buuren, J. P.L. Brand, C. G.M. Groothuis-Oudshoorn, and D. B. Rubin. Fully conditional specification in multivariate imputation. *Journal of Statistical Computation and Simulation*, 76(12):1049–1064, 2006. ISSN 15635163, 00949655. doi: 10.1080/10629360600810434.

S. Van Buuren. *Flexible Imputation of Missing Data*. CRC Press, second edition edition, 2018.

Rosemary Tawn, Jethro Browell, and Iain Dinwoodie. Missing data in wind farm time series: Properties and effect on forecasts. *Electric Power Systems Research*, 189: 106640, 2020. ISSN 18732046, 03787796. doi: 10.1016/j.epsr.2020.106640.

Yi He, Christian Schreckenberger, Heiner Stuckenschmidt, and Xindong Wu. Towards utilitarian online learning - a review of online algorithms in open feature space. *Ijcai*

International Joint Conference on Artificial Intelligence, 2023-:6647–6655, 2023. ISSN 10450823.

Masashi Sugiyama. Chapter 12 - maximum likelihood estimation. *Introduction To Statistical Machine Learning*, pages 123–138, 2016. doi: 10.1016/B978-0-12-802121-7.00023-6.

Roger Grosse. Csc 421/2516 winter 2019: Lecture 2, part 3: Training a classifier, 2019.

Frank E. Harrell. Missing data. *Regression Modeling Strategies*, pages 41–52, 2001. doi: 10.1007/978-1-4757-3462-1_3.

Nicholas T. Longford. Multiple imputation. *Missing Data and Small-area Estimation*, pages 59–95, 2005. doi: 10.1007/1-84628-195-4_4.

Francesco Orabona. A modern introduction to online learning. 2023.

Tim Roughgarden. Lectures notes on algorithmic game theory stanford cs364a, fall 20131, 7 2014.

Vincent Knight. Lecture notes for game theory, chapter 16 - cooperative games, 2015.

Yi He, Baijun Wu, Di Wu, Ege Beyazit, Sheng Chen, and Xindong Wu. Online learning from capricious data streams: A generative approach. *Ijcai International Joint Conference on Artificial Intelligence*, 2019-:2491–2497, 2019. ISSN 10450823. doi: 10.24963/ijcai.2019/346.

Ege Beyazit, Jeevithan Alagurajah, and Xindong Wu. Online learning from data streams with varying feature spaces. *33rd Aaai Conference on Artificial Intelligence, Aaai 2019, 31st Innovative Applications of Artificial Intelligence Conference, Iaai 2019 and the 9th Aaai Symposium on Educational Advances in Artificial Intelligence, Eaai 2019*, pages 3232–3239, 2019. ISSN 23743468, 21595399.

Yi He, Jiaxian Dong, Bo-Jian Hou, Yu Wang, and Fei Wang. Online learning in variable feature spaces with mixed data. *2021 Ieee International Conference on Data Mining (icdm)*, pages 181–190, 2021. ISSN 23748486, 15504786. doi: 10.1109/ICDM51629.2021.00028.

Kolby Nottingham Markelle Kelly, Rachel LongJohn. The uci machine learning repository. URL https://archive.ics.uci.edu.

Caroline Draxl, Andrew Clifton, Bri-Mathias Hodge, and Jim McCaa. The wind integration national dataset (wind) toolkit. *Applied Energy*, 151:355–366, 2015.

Thomas Falconer, Jalal Kazempour, and Pierre Pinson. Bayesian regression markets. 2024b.

Have you ever wondered why data is always siloed up? Have you been there, screaming for help, but no one wanted to give you the data you so desperately needed? They wouldn't even sell it to you, "oh, it's so hard to put a price on data", they would say. You even presented them with the regression market, said, "here, we've solved it. We can price data." They simply said "what if I want to leave the market again? What if I want my friend Tim to join the market? What then?" Well, I have the solution for you! Read this thesis and find out how to build a regression market where participants can freely enter and leave the market, no muss no fuss.

Technical
University of
Denmark

Frederiksborgvej 399
4000 Roskilde
Tlf. 4525 1700

www.vindenergi.dtu.dk